

What is Angular?

Angular is a **TypeScript-based open-source front-end framework** used to build **single-page applications (SPAs)** and dynamic web applications. It is developed and maintained by **Google**.

Angular helps developers create **structured, scalable, and maintainable client-side applications** by providing a complete framework with built-in tools for routing, form handling, HTTP communication, and dependency management.

In simple terms:

Angular is a framework that helps developers build modern, dynamic web applications with a component-based architecture.

Instead of manually manipulating the DOM like in traditional JavaScript or jQuery, Angular uses **declarative templates and components** to manage UI and application logic.

Key Features of Angular

1. Component-Based Architecture

Angular applications are built using **components**.

A **component** controls a portion of the UI and contains:

- HTML template (view)
- TypeScript class (logic)

- CSS styles

Example:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  template: `<h1>Hello {{name}}</h1>`
})
export class HelloComponent {
  name = "Angular";
}
```

Benefits:

- Code reusability
- Easy maintenance
- Better modular design

2. Two-Way Data Binding

Angular supports **two-way data binding**, meaning:

- Changes in UI update the model.
- Changes in the model update the UI automatically.

Example:

```
<input [(ngModel)]="username">
<p>Hello {{username}}</p>
```

If the user types in the input field, the paragraph updates instantly.

Benefits:

- Synchronizes UI and data automatically
- Reduces manual DOM manipulation

3. Dependency Injection (DI)

Angular has a **built-in Dependency Injection system** that helps manage services efficiently.

Example:

```
constructor(private userService: UserService) {}
```

Benefits:

- Loose coupling
- Better testability
- Easier code maintenance

4. TypeScript Support

Angular is built using **TypeScript**, which is a superset of JavaScript.

Advantages of TypeScript:

- Static typing
- Better IDE support
- Easier debugging
- Object-oriented features (classes, interfaces)

Example:

```
interface User {  
  id: number;  
  name: string;  
}
```

5. Directives

Directives allow developers to **extend HTML functionality**.

Types of directives:

1. **Structural directives** – change DOM layout
 - a. *ngIf
 - b. *ngFor
2. **Attribute directives** – change element behavior
 - a. ngClass
 - b. ngStyle

Example:

```
<div *ngIf="isLoggedIn">Welcome User</div>
```

6. Routing

Angular provides a **powerful routing module** for navigation between views.

Example:

```
const routes: Routes = [  
  { path: 'login', component: LoginComponent },  
  { path: 'dashboard', component:  
DashboardComponent }  
];
```

Benefits:

- Enables **Single Page Application navigation**
- No full page reload

7. Services and Reusability

Angular uses **services** to share logic across components.

Example:

```
@Injectable()  
export class AuthService {  
  login() {  
    return "User logged in";  
  }  
}
```

Services are commonly used for:

- API calls
- Business logic
- Data sharing

8. Built-in HTTP Client

Angular provides **HttpClient** for communicating with backend APIs.

Example:

```
this.http.get('https://api.example.com/users')  
  .subscribe(data => console.log(data));
```

This makes it easy to integrate with backend frameworks like:

- Spring Boot
- Node.js
- .NET

9. Modular Architecture

Angular applications are divided into **modules**.

Example:

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule],  
  bootstrap: [AppComponent]  
})  
export class AppModule {}
```

Benefits:

- Better organization
- Lazy loading support

- Improved scalability

10. Built-in Testing Support

Angular supports testing using:

- **Karma**
- **Jasmine**

This allows developers to write **unit tests and integration tests** easily.

Simple Definition for Interviews

Angular is a TypeScript-based front-end framework developed by Google that is used to build scalable, dynamic, single-page web applications using components, data binding, dependency injection, and modular architecture.

Why were client-side frameworks like Angular introduced?

Client-side frameworks like **Angular** were introduced to solve the problems developers faced when building large, dynamic web applications using plain **JavaScript** or libraries like **jQuery**.

Earlier websites were mostly **static pages**, but modern applications require complex UI interactions, real-time updates, and large codebases. Managing this with only JavaScript or jQuery became difficult.

Below are the main reasons why frameworks like Angular were introduced.

1. To Manage Growing Application Complexity

In early web development, developers used:

- HTML for structure
- CSS for styling
- JavaScript/jQuery for DOM manipulation

Example with jQuery:

```
$("#btn").click(function(){  
    $("#message").text("Welcome");  
});
```

When applications became large (hundreds of screens and features):

- JavaScript files became huge
- Logic was scattered everywhere
- Difficult to maintain and debug

Angular introduced a **structured architecture** using:

- Components
- Modules
- Services

This makes large applications easier to organize.

2. Separation of Concerns

Before frameworks, developers often mixed:

- UI logic
- business logic
- DOM manipulation

Everything was inside the same JavaScript file.

Angular introduced **clear separation**:

Layer	Responsibility
Component	Application logic
Template	UI (HTML)
Service	Business logic / API calls

This makes code easier to maintain and scale.

3. Reduce Manual DOM Manipulation

With jQuery or vanilla JavaScript, developers manually update the DOM.

Example:

```
document.getElementById("name").innerHTML = userName;
```

In large apps this leads to:

- complex code
- difficult debugging

Angular introduced **data binding**, where UI updates automatically.

Example:

```
<p>{{username}}</p>
```

When the value changes in the component, Angular updates the DOM automatically.

4. Reusable Components

Without frameworks, developers often copied and pasted code.

Angular introduced **component-based architecture**.

Example components:

- LoginComponent
- NavbarComponent
- ProductComponent
- DashboardComponent

Each component can be reused across the application.

Benefits:

- cleaner code
- faster development
- easier testing

5. Better State and Data Management

Modern applications need to handle:

- user data
- API responses
- application state

Frameworks provide structured ways to manage this data.

Angular provides:

- services
- dependency injection
- RxJS for reactive programming

6. Single Page Application (SPA) Support

Older websites reloaded the entire page for navigation.

Example:

Home → reload

Products → reload

Contact → reload

Frameworks like Angular allow **Single Page Applications (SPA)**.

Navigation happens without full page reload using Angular routing.

Example:

Home → Products → Cart

All handled in the browser.

Benefits:

- faster navigation
- smoother user experience

7. Built-in Tools and Features

Angular provides many built-in features developers otherwise had to build manually:

- Routing
- Form validation
- HTTP client
- Dependency injection
- Testing support
- CLI tools

This improves development productivity.

Short Interview Answer

You can answer like this in interviews:

Client-side frameworks like Angular were introduced to solve the challenges of building large and complex web applications using plain JavaScript or jQuery. They provide a structured architecture,

component-based development, data binding, reusable components, and built-in tools for routing, forms, and API communication, making applications easier to develop, maintain, and scale.

Application logic vs bussiness logic: or service vs component:

UI (Template) → Application Logic (Component) → Business Logic (Service) → Backend/API

Let's understand clearly.

1. Application Logic (Component Logic)

Application logic is the logic that **controls how the UI behaves and interacts with the application.**

It mainly handles:

- User interactions (click, submit, change)
- Calling services
- Preparing data for the UI
- Managing UI state (loading, error, show/hide)

This logic usually lives inside **Angular Components.**

Example

Component:

```
export class LoginComponent {  
  
    username = '';  
    password = '';  
  
    constructor(private authService: AuthService) {}  
  
    login() {  
        this.authService.login(this.username,  
this.password);  
    }  
}
```

Here the component:

- gets values from UI
- calls the service
- controls the flow

So this is **application logic**.

2. Business Logic (Domain Logic)

Business logic represents the **actual rules of the business or system**.

These rules define **how the system should behave according to business requirements**.

Examples of business rules:

- User must be **18+ to register**
- Discount is **10% for premium users**
- Order cannot be placed if **stock = 0**
- Password must be **minimum 8 characters**

This logic usually lives in **Services**.

Example

Service:

```
export class AuthService {  
  login(username: string, password: string) {  
    if(password.length < 8){  
      throw new Error("Password must be at least 8  
characters");  
    }  
  
    return this.http.post('/api/login', {username,  
password});  
  }  
}
```

This rule:

Password must be at least 8 characters

is **business logic**.

3. Simple Real Example (E-commerce)

Application Logic (Component)

```
placeOrder(){  
    this.orderService.placeOrder(this.cartItems);  
}
```

Responsibilities:

- Get cart items
- Call service
- Show success message

Business Logic (Service)

```
placeOrder(items){  
  
    if(items.length === 0){  
        throw new Error("Cart cannot be empty");  
    }  
  
    return this.http.post('/api/orders', items);  
}
```

Business rule:

Cart cannot be empty

4. Simple Difference

Application Logic	Business Logic
Controls UI behavior	Implements business rules
Lives in Components	Lives in Services
Handles user interaction	Handles system rules
Connects UI and services	Contains core domain logic

How does an Angular application work?

1. Application Bootstrapping (Application Start)

When an Angular application starts, the **main entry file** is:

`main.ts`

Example:

```
platformBrowserDynamic().bootstrapModule(AppModule);
```

Here Angular **bootstraps** the root module called AppModule.

Bootstrapping means:

Starting the Angular application.

2. Root Module (AppModule)

After bootstrapping, Angular loads the **root module**.

Example:

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule],  
  bootstrap: [AppComponent]  
})  
export class AppModule {}
```

Responsibilities of a module:

- Declares components
- Imports other modules
- Defines the starting component

The **bootstrap property** tells Angular which component should start the application.

3. Root Component Loads

Angular then loads the root component:

AppComponent

Example:

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html'  
})  
export class AppComponent {}
```

The **selector**:

```
<app-root></app-root>
```

is placed in **index.html**.

So Angular replaces this tag with the component UI.

4. Template Rendering (View Creation)

Angular reads the **component template (HTML)** and renders it in the browser.

Example:

```
<h1>Welcome to Angular App</h1>
```

This is the **View (UI)**.

Angular connects:

Component (Logic)



Template (HTML View)

5. Data Binding

Angular synchronizes data between the **component** and the **UI** using **data binding**.

Example:

```
export class AppComponent {  
  title = "Angular Application";  
}
```

Template:

```
<h1>{{title}}</h1>
```

Angular automatically updates the UI if the value changes.

6. Event Handling

Angular listens to user actions like:

- click
- input
- submit

Example:

```
<button (click)="sayHello()">Click</button>
```

Component:

```
sayHello(){  
  alert("Hello!");  
}
```

Flow:

User Click → Component Method → UI Update

7. Services and Dependency Injection

For reusable logic like:

- API calls
- business logic
- data sharing

Angular uses **services**.

Example:

```
constructor(private userService: UserService){}
```

Angular automatically provides the service using **Dependency Injection**.

8. Routing (Navigation Between Pages)

Angular uses **routing** to navigate between views without reloading the page.

Example:

```
const routes: Routes = [  
  { path: 'login', component: LoginComponent },  
  { path: 'dashboard', component:  
DashboardComponent }  
];
```

This enables **Single Page Applications (SPA)**.

9. Change Detection

Angular automatically detects when data changes.

Example:

```
this.title = "New Title";
```

Angular updates the DOM automatically.

You don't need manual DOM manipulation like in **jQuery**.

10. Communication With Backend APIs

Angular communicates with backend servers using the HTTP client.

Example:

```
this.http.get('/api/users').subscribe(data => {  
    this.users = data;  
});
```

Angular applications commonly connect with backends like:

- **Spring Boot**
- Node.js
- .NET

Complete Flow of Angular Application

index.html



main.ts



AppModule



AppComponent



Component + Template



User Interaction



Services / API Calls



Change Detection Updates UI

Short Interview Answer

You can answer like this:

An Angular application starts by bootstrapping the root module in `main.ts`. Angular then loads the root module and initializes the root component. The component renders its template in the browser, and Angular binds data between the component and the view. User interactions trigger component methods, which may call services to perform business logic or API calls. Angular's change detection mechanism automatically updates the UI when data changes.

What are the building blocks of Angular?

The main building blocks of **Angular** are the fundamental parts used to create and organize an Angular application. These blocks help structure the application in a modular and maintainable way.

The core building blocks of Angular are:

1. Modules
2. Components
3. Templates
4. Directives
5. Services
6. Dependency Injection
7. Routing
8. Data Binding

Let's understand each one clearly.

1. Modules

An Angular **module** groups related components, directives, and services together.

Every Angular application has at least one root module called **AppModule**.

Example:

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Responsibilities of modules:

- Organize the application
- Register components
- Import other modules
- Define the starting component

Example modules in real apps:

- UserModule
- ProductModule
- AdminModule

2. Components

A **component** is the most important building block in Angular.

It controls a part of the user interface.

Each component contains:

- HTML template (UI)
- TypeScript class (logic)
- CSS styles

Example:

```
@Component({
  selector: 'app-user',
  template: `<h2>{{name}}</h2>`
})
export class UserComponent {
  name = "Lakshmi";
}
```

Here:

- UserComponent → logic
- template → UI

3. Templates

Templates define the **view (UI)** of a component using HTML.

They display data and bind UI elements with component properties.

Example:

```
<h1>{{title}}</h1>  
<button (click)="save()">Save</button>
```

Templates support:

- data binding
- event binding
- directives

4. Directives

Directives extend the behavior of HTML elements.

Angular has three types of directives.

Structural Directives

They change the structure of the DOM.

Examples:

*ngIf

*ngFor

Example:

```
<div *ngIf="isLoggedIn">Welcome</div>
```

Attribute Directives

They change the appearance or behavior of elements.

Examples:

ngClass

ngStyle

5. Services

Services are used to store **business logic and reusable functionality**.

They are commonly used for:

- API calls
- data sharing
- reusable logic

Example:

```
@Injectable()
export class UserService {

  getUsers(){
    return this.http.get('/api/users');
  }
}
```

```
}
```

Components call services when needed.

6. Dependency Injection (DI)

Angular uses **dependency injection** to provide services to components automatically.

Example:

```
constructor(private userService: UserService) {}
```

Angular creates and injects the service instance.

Benefits:

- loose coupling
- easier testing
- reusable services

7. Routing

Routing allows navigation between different components without reloading the page.

Example:

```
const routes: Routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'login', component: LoginComponent }  
]
```

```
];
```

This enables **Single Page Applications (SPA)**.

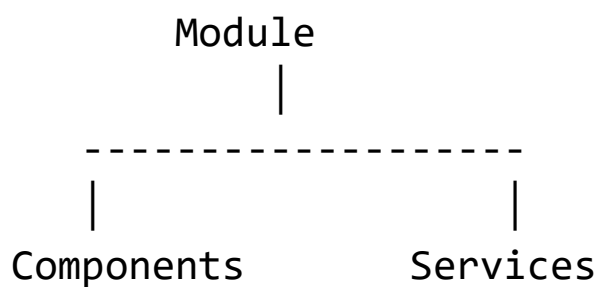
8. Data Binding

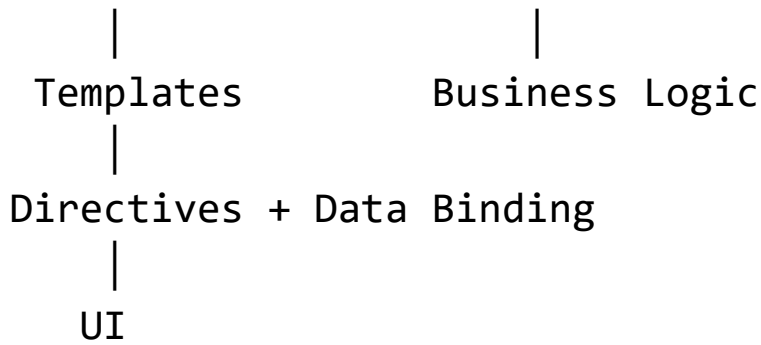
Data binding connects the **component data** with the **template UI**.

Types of data binding:

Type	Example
Interpolation	{{name}}
Property Binding	[src]="imageUrl"
Event Binding	(click)="save()"
Two-way Binding	[(ngModel)]="username"

Simple Diagram of Angular Building Blocks





Short Interview Answer

You can answer like this:

The main building blocks of Angular are modules, components, templates, directives, services, dependency injection, routing, and data binding. Modules organize the application, components control the UI, templates define the view, directives modify DOM behavior, services contain business logic, dependency injection manages service instances, routing enables navigation, and data binding synchronizes data between the component and the view.

What is the difference between Angular and AngularJS?

The difference between **Angular** and **AngularJS** is mainly in **architecture, performance, language support, and modern features**. AngularJS is the older framework, while Angular is the completely redesigned modern version.

Below is a clear comparison.

1. Release and Version

AngularJS	Angular
Released in 2010	Released in 2016
Also called Angular 1.x	Called Angular 2+
Older framework	Modern framework

Angular is **not just an upgrade**, it is a **complete rewrite** of AngularJS.

2. Programming Language

AngularJS	Angular
Uses JavaScript	Uses TypeScript
No strong typing	Supports static typing
Harder to maintain large apps	Better for large-scale applications

Example:

AngularJS:


```
$scope.name = "Lakshmi";
```

Angular:

```
name: string = "Lakshmi";
```

3. Architecture

AngularJS	Angular
Based on MVC architecture	Based on Component-based architecture
Controllers manage logic	Components manage UI and logic

Example:

AngularJS uses:

Controller

Scope

Angular uses:

Component

Template

Service

Module

Component-based architecture makes Angular easier to scale.

4. Performance

AngularJS	Angular
Slower performance	Much faster
Uses dirty checking	Uses change detection mechanism

Angular also supports:

- Ahead-of-Time (AOT) compilation
- Faster rendering

5. Mobile Support

AngularJS	Angular
Not optimized for mobile	Designed for modern mobile apps

Angular works well with:

- Progressive Web Apps (PWA)
- Mobile-first development

6. Dependency Injection

AngularJS	Angular
Basic dependency injection	Advanced dependency injection system

Angular provides better service management.

7. Structure and Maintainability

AngularJS	Angular
Harder to maintain large applications	Better modular structure
Less organized	Uses modules and components

Angular applications are easier to scale in enterprise projects.

8. Tooling and CLI

Angular provides a powerful tool:

Angular CLI

Example:

```
ng new my-app
ng generate component user
```

AngularJS does not provide such built-in CLI support.

Simple Difference Table

Feature	AngularJS	Angular
Release	2010	2016
Language	JavaScript	TypeScript
Architecture	MVC	Component-based
Performance	Slower	Faster
Mobile support	Limited	Strong
CLI	No	Yes
Maintenance	Difficult for large apps	Easier for large apps

Short Interview Answer

You can answer like this:

AngularJS is the older JavaScript-based framework released in 2010 that follows MVC architecture. Angular, released in 2016, is a completely redesigned framework built with TypeScript and uses a component-based architecture. Angular provides better performance, improved dependency injection, modular development, and modern features like Angular CLI and mobile support.

What is a Component in Angular?

A **Component** is the **fundamental building block of an Angular application**. In **Angular**, a component controls a **part of the user interface (UI)** and contains the logic required for that part.

In simple terms:

A component is a TypeScript class that controls a specific section of the UI using an HTML template.

Each Angular application is built by combining multiple components.

Example parts of a website:

- Navbar
- Login form
- Product list

- Dashboard

Each of these can be created as a **separate component**.

Structure of a Component

A component mainly consists of three parts:

Part	Purpose
TypeScript Class	Contains application logic
HTML Template	Defines the UI
CSS Styles	Defines styling

Example:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `<h2>{{name}}</h2>`,
  styles: [`h2 { color: blue; }`]
})
export class UserComponent {
  name = "Lakshmi";
}
```

Explanation:

- **selector** → HTML tag used to display the component
- **template** → HTML view
- **class** → component logic

How a Component Works

Step-by-step flow:

1. Angular loads the component.
2. The **template (HTML)** is rendered in the browser.
3. The **component class provides data and logic**.
4. Angular binds data between the component and template.

Example:

Component:

```
export class AppComponent {  
  title = "Angular Application";  
}
```

Template:

```
<h1>{{title}}</h1>
```

Angular automatically displays:

Angular Application

Component Example in Real Application

Example: **Login Component**

Component:

```
export class LoginComponent {  
  username = '';  
  password = '';  
  
  login(){  
    console.log("Login clicked");  
  }  
}
```

Template:

```
<input [(ngModel)]="username">  
<input [(ngModel)]="password" type="password">  
<button (click)="login()">Login</button>
```

Responsibilities of this component:

- capture user input
- handle login action
- interact with services

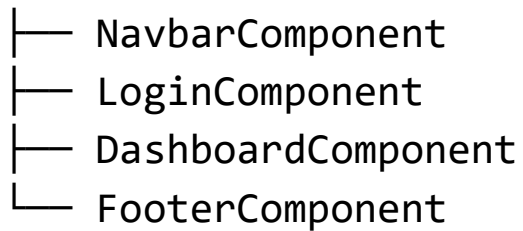
Why Components are Important

Components help:

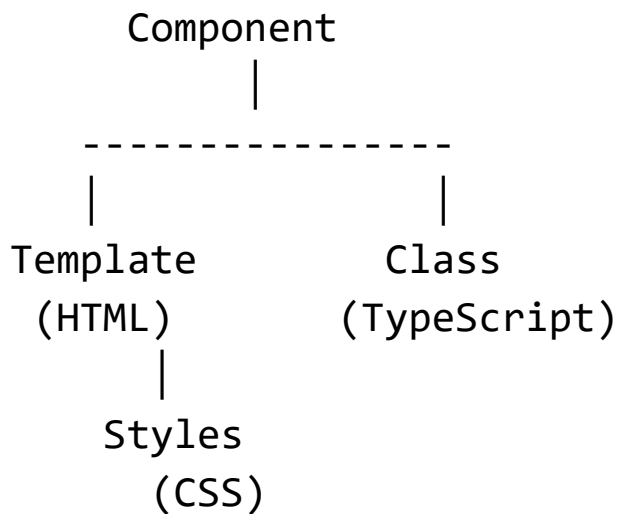
- divide UI into **small reusable parts**
- improve **maintainability**
- enable **modular architecture**

Example application structure:

AppComponent



Simple Diagram



Short Interview Answer

You can answer like this:

A component in Angular is a fundamental building block that controls a part of the user interface. It consists of a TypeScript class that contains the logic, an HTML template that defines the view, and optional CSS styles. Components enable modular and reusable UI development in Angular applications.

What is a Module in Angular?

In **Angular**, a **Module** is a container that groups related components, directives, pipes, and services together. It helps organize the application into logical sections and makes the application easier to maintain and scale.

In Angular, modules are defined using the `@NgModule` **decorator**.

In simple terms:

A module is a logical container that organizes and manages different parts of an Angular application.

Why Modules are Needed

As an Angular application grows, it may contain many:

- Components
- Services
- Directives
- Pipes

Managing everything in one place becomes difficult. Modules help **group related functionality together**.

Example structure:

```
AppModule
├── UserModule
├── ProductModule
└── AdminModule
```

Each module handles a specific feature of the application.

Root Module (AppModule)

Every Angular application has a **root module** called AppModule. This is the first module that Angular loads when the application starts.

Example:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
```

```
})  
export class AppModule {}
```

Explanation:

Property	Purpose
declarations	Registers components, directives, and pipes
imports	Imports other modules
providers	Registers services
bootstrap	Defines the starting component

Types of Angular Modules

1. Root Module

The main module that starts the application.

Example:

`AppModule`

2. Feature Modules

Modules created for specific features of the application.

Example:

UserModule

ProductModule

AdminModule

These improve modularity and maintainability.

3. Shared Module

Contains reusable components, directives, and pipes used across multiple modules.

Example:

SharedModule

Contains things like:

- NavbarComponent
- FooterComponent

4. Core Module

Contains singleton services used across the application.

Example services:

- AuthenticationService
- LoggerService

Simple Working Flow

main.ts



AppModule (Root Module)



Components

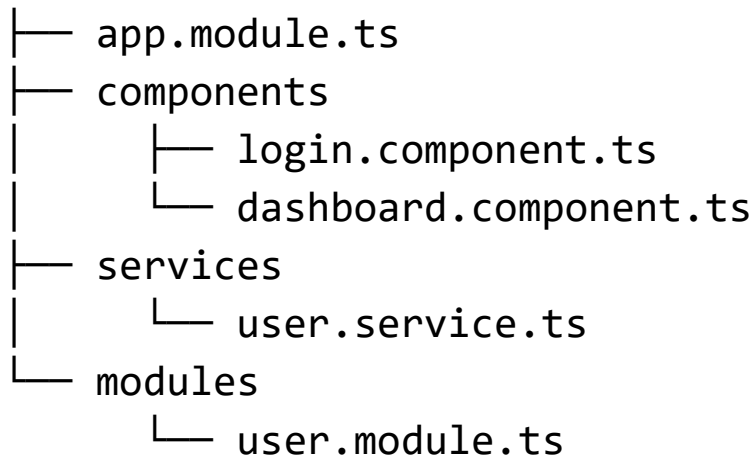


Templates (UI)

Angular first loads the **root module**, which then loads the components required for the application.

Example Real Application Structure

src/app



Short Interview Answer

You can answer like this:

A module in Angular is a container that groups related components, directives, pipes, and services together. Modules help organize the application into logical sections and improve maintainability. Every Angular application has a root module called AppModule that bootstraps the application.

What is the purpose of App Module?

In **Angular**, **AppModule** is the **root module of the Angular application**. It is the module that Angular loads first when the application starts.

The main purpose of **AppModule** is to **configure and initialize the application by bringing together components, services, and other modules**.

In simple terms:

AppModule is the main module that starts and organizes the Angular application.

1. Bootstraps the Application

AppModule tells Angular **which component should start the application**.

Example:

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule],  
  bootstrap: [AppComponent]
```

```
})  
export class AppModule {}
```

Here:

```
bootstrap: [AppComponent]
```

means:

Angular starts the application with AppComponent

So AppComponent becomes the root component.

2. Declares Components, Directives, and Pipes

AppModule registers all the components used in the application.

Example:

```
declarations: [  
  AppComponent,  
  LoginComponent,  
  DashboardComponent  
]
```

This tells Angular that these components belong to this module.

3. Imports Other Modules

AppModule imports required Angular modules.

Example:

```
imports: [  
  BrowserModule,  
  FormsModule,  
  HttpClientModule  
]
```

These modules provide important features:

- **BrowserModule** → required to run Angular in a browser
- **FormsModule** → form handling
- **HttpClientModule** → API communication

4. Provides Services

AppModule can register services so they can be used across the application.

Example:

```
providers: [UserService]
```

This allows components to use the service through **dependency injection**.

5. Acts as the Central Configuration

AppModule works as the **central configuration point** of the Angular application.

It connects:

- components
- services
- modules
- application startup

Simple Application Flow

main.ts



AppModule



AppComponent



Other Components

Angular first loads **AppModule**, then it loads the **root component**.

Short Interview Answer

You can answer like this:

The purpose of AppModule in Angular is to act as the root module that initializes the application. It declares components, imports required modules, provides services, and bootstraps the root component that starts the application.

What is the Angular CLI and why is it used?

Angular CLI (Command Line Interface) is a command-line tool used to **create, develop, build, and manage Angular applications easily**. It automates many development tasks so developers don't have to manually configure the project.

In simple terms:

Angular CLI is a tool that helps developers quickly create and manage Angular projects using simple commands.

It is used from the terminal or command prompt.

Example command:

```
ng new my-app
```

This command automatically creates a **complete Angular project structure**.

Why Angular CLI is Used

Angular applications involve many configurations such as:

- project structure
- TypeScript configuration
- Webpack build setup
- testing configuration
- development server

Doing this manually would take a lot of time. Angular CLI **automates all these tasks.**

1. Creating Angular Projects Quickly

Angular CLI can generate a complete project with a single command.

Example:

```
ng new my-angular-app
```

This automatically creates:

```
src/  
app/  
assets/  
angular.json  
package.json  
tsconfig.json
```

So developers don't need to manually set up the project.

2. Generating Application Components

Angular CLI can generate components, services, modules, and more.

Example:

Create a component:

```
ng generate component login
```

or

```
ng g c login
```

Angular CLI automatically creates:

```
login.component.ts  
login.component.html  
login.component.css  
login.component.spec.ts
```

and also **updates the module automatically**.

3. Running the Development Server

Angular CLI provides a built-in development server.

Example:

```
ng serve
```

This starts the application at:

<http://localhost:4200>

It also supports **live reload**, meaning the browser refreshes automatically when code changes.

4. Building the Application

Angular CLI can compile the application for production.

Example:

```
ng build
```

Production build:

```
ng build --prod
```

This creates optimized files for deployment.

5. Running Tests

Angular CLI supports testing using tools like **Karma** and **Jasmine**.

Example:

```
ng test
```

6. Managing the Project Structure

Angular CLI ensures a **standard project structure**, which helps teams maintain consistency.

Example structure:

```
src/  
├─ app/  
│   └─ components/
```

```
|
|   ├── services/
|   └── app.module.ts
└── assets/
    └── environments/
```

Common Angular CLI Commands

Command	Purpose
ng new app-name	Create new Angular project
ng serve	Run development server
ng generate component	Create component
ng generate service	Create service
ng build	Build the application
ng test	Run tests

Short Interview Answer

You can answer like this:

Angular CLI is a command-line tool used to create, develop, build, and manage Angular applications. It helps automate tasks such as project creation, generating components and services, running the development server, building the application, and executing tests.

What is TypeScript and why does Angular use it?

TypeScript is an open-source programming language developed by **Microsoft**. It is a **superset of JavaScript**, which means it includes all JavaScript features plus additional features like **static typing, interfaces, and object-oriented programming support**.

TypeScript code is compiled (transpiled) into JavaScript so it can run in any browser.

In simple terms:

TypeScript is JavaScript with additional features that make code easier to write, understand, and maintain.

Why Angular Uses TypeScript

Angular uses TypeScript because it provides features that help build **large and maintainable applications**.

Below are the main reasons.

1. Static Typing

TypeScript allows developers to define **data types for variables, parameters, and return values**.

Example:

```
let age: number = 25;  
let name: string = "Lakshmi";
```

Benefit:

- Errors are detected during development instead of runtime.
- Improves code reliability.

Example error detection:

```
let age: number = "twenty";
```

This will produce a compile-time error.

2. Object-Oriented Programming (OOP)

TypeScript supports OOP features such as:

- Classes
- Interfaces
- Inheritance
- Access modifiers

Example:

```
class User {  
  name: string;  
  
  constructor(name: string) {  
    this.name = name;  
  }  
}
```

Angular heavily relies on **classes for components, services, and modules**.

3. Interfaces

Interfaces help define the structure of objects.

Example:

```
interface User {  
  id: number;  
  name: string;  
}
```

This ensures the object always follows a defined structure.

This is very useful when working with **API responses**.

4. Better Code Organization

TypeScript helps structure large applications better.

Angular applications can contain:

- hundreds of components
- multiple services
- complex business logic

TypeScript makes this easier to manage using **classes, modules, and interfaces**.

5. Better IDE Support

TypeScript provides strong support in IDEs like **Visual Studio Code**.

Features include:

- Auto-completion
- Error detection
- Refactoring support
- Code navigation

This improves developer productivity.

6. Early Error Detection

TypeScript catches many errors during compilation instead of runtime.

Example:

```
function add(a: number, b: number) {  
    return a + b;  
}
```

```
add(5, "10");
```

TypeScript will show an error before running the code.

7. Supports Modern JavaScript Features

TypeScript supports modern JavaScript features like:

- ES6 modules
- arrow functions
- async/await
- decorators

Angular uses **decorators** such as:

```
@Component()  
@Injectable()  
@NgModule()
```

These are important Angular concepts.

Simple Comparison

JavaScript	TypeScript
Dynamic typing	Static typing

Fewer development tools	Better IDE support
Harder to maintain large apps	Better for large applications
Runtime errors more common	Early error detection

Short Interview Answer

You can answer like this:

TypeScript is a superset of JavaScript developed by Microsoft that adds features like static typing, classes, interfaces, and object-oriented programming support. Angular uses TypeScript because it improves code maintainability, provides better tooling support, enables early error detection, and helps structure large-scale applications more effectively.

Angular Components & Architecture

In **Angular**, applications are built using a **component-based architecture**. This architecture divides the application into small, reusable parts called **components**, which work together with modules, services, templates, and other Angular features.

Understanding **Angular components and architecture** helps developers build scalable and maintainable applications.

1. Angular Architecture Overview

An Angular application is organized in layers. The main architectural flow looks like this:

```
User Interface (Template)
      ↓
Component (Application Logic)
      ↓
Service (Business Logic)
      ↓
Backend API
```

Or in a more complete Angular structure:

```
Module
├── Components
│   └── Templates (HTML View)
├── Services
├── Directives
└── Pipes
```

2. Angular Component

A **component** is the **basic building block of an Angular application**.

A component controls a **portion of the UI** and contains the logic needed for that UI.

A component consists of three main parts:

Part	Description
TypeScript Class	Contains logic and data
HTML Template	Defines the UI
CSS Styles	Defines the design

Example component:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `<h2>{{name}}</h2>`
})
export class UserComponent {
  name = "Lakshmi";
}
```

Explanation:

- `@Component` → decorator defining the component
- `selector` → HTML tag used to display the component
- `template` → UI view
- `class` → component logic

Usage in HTML:

```
<app-user></app-user>
```

3. Component Responsibilities

A component mainly handles:

- UI logic
- user interactions
- calling services
- data binding with templates

Example:

```
export class LoginComponent {  
  
  username = '';  
  
  login(){  
    console.log("User logged in");  
  }  
}
```



```
}
```

Template:

```
<input [(ngModel)]="username">  
<button (click)="login()">Login</button>
```

Here the component:

- captures input
- processes logic
- updates UI

4. Angular Application Architecture

Angular applications follow a **modular architecture**.

Main building blocks include:

1. Modules

Modules organize the application into logical sections.

Example:

```
AppModule  
UserModule  
AdminModule
```

The root module is **AppModule**.

2. Components

Components manage specific UI parts.

Example:

NavbarComponent

LoginComponent

DashboardComponent

FooterComponent

3. Templates

Templates define the **HTML view**.

Example:

```
<h1>{{title}}</h1>
```

4. Services

Services contain **business logic and reusable functionality**.

Example:

```
@Injectable()
export class UserService {

  getUsers(){
    return this.http.get('/api/users');
  }
}
```

```
}
```

5. Dependency Injection

Angular uses **Dependency Injection** to provide services to components.

Example:

```
constructor(private userService: UserService) {}
```

Angular automatically injects the service instance.

6. Directives

Directives extend HTML behavior.

Examples:

```
*ngIf  
*ngFor  
ngClass
```

Example:

```
<div *ngIf="isLoggedIn">Welcome</div>
```

7. Pipes

Pipes transform data before displaying it in the UI.

Example:

```
{{today | date}}
```

8. Routing

Routing allows navigation between views without reloading the page.

Example:

```
const routes: Routes = [  
  { path: 'login', component: LoginComponent }  
];
```

5. Complete Angular Architecture Flow

```
index.html  
  ↓  
main.ts  
  ↓  
AppModule  
  ↓  
AppComponent  
  ↓  
Other Components  
  ↓  
Services  
  ↓  
Backend API
```

6. Example Real Application Structure

```
src/app
├── app.module.ts
├── app.component.ts
├── components
│   ├── login
│   ├── dashboard
│   └── navbar
├── services
│   └── user.service.ts
└── models
    └── user.model.ts
```

Short Interview Answer

You can say:

Angular uses a component-based architecture where the application is divided into reusable components. Each component consists of a TypeScript class for logic, an HTML template for the view, and optional CSS for styling. Components are organized inside modules, and services are used to handle business logic. Angular also uses dependency injection, directives, pipes, and routing to build scalable single-page applications.

What is a component decorator in Angular?

In **Angular**, a **Component Decorator** is a special function used to define metadata for a component. It tells Angular **how the component should be created, configured, and used in the application**.

The component decorator used in Angular is `@Component`.

In simple terms:

A component decorator provides configuration details that tell Angular how a component should behave and how its UI should be rendered.

1. What is @Component Decorator?

The `@Component` **decorator** is placed above a TypeScript class to convert it into an Angular component.

Example:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `<h2>Hello {{name}}</h2>`,
  styleUrls: ['./user.component.css']
})
export class UserComponent {
```

```
name = "Lakshmi";  
}
```

Here:

- `@Component` → decorator
- `UserComponent` → component class

The decorator contains **metadata** about the component.

2. Common Properties of Component Decorator

The `@Component` decorator includes several important properties.

Property	Purpose
selector	Defines the HTML tag used to display the component
template	Defines the HTML view directly
templateUrl	Path to external HTML file
styleUrls	Path to CSS file
providers	Services used by the component

3. Example with External Template

Component:

```
@Component({  
  selector: 'app-login',  
  templateUrl: './login.component.html',  
  styleUrls: ['./login.component.css']  
})  
export class LoginComponent {}
```

HTML:

```
<h2>Login Page</h2>
```

Usage in HTML:

```
<app-login></app-login>
```

Angular replaces this tag with the component UI.

4. How Component Decorator Works

Flow:

@Component decorator

↓

Defines metadata

↓

Angular reads metadata



Creates and renders component

The decorator tells Angular:

- what the component selector is
- where the template is located
- what styles to apply

5. Why Component Decorator is Important

Without the decorator, Angular will treat the class as a normal TypeScript class.

The decorator makes Angular understand:

This class is a component

and it should be part of the UI.

6. Example with Real Angular Flow

Component:

```
@Component({
  selector: 'app-dashboard',
  template: `<h1>Dashboard</h1>`
})
export class DashboardComponent {}
```

In HTML:

```
<app-dashboard></app-dashboard>
```

Angular renders:

Dashboard

Short Interview Answer

You can say:

In Angular, a component decorator is the `@Component` decorator used to define metadata for a component. It tells Angular how the component should be created and used by specifying properties such as selector, template, and styles. It converts a TypeScript class into an Angular component that controls a part of the UI.

What is component metadata?

In **Angular**, **component metadata** is the **configuration information provided inside the `@Component` decorator** that tells Angular **how the component should behave, how its UI should be rendered, and how it should be used in the application.**

In simple terms:

Component metadata is the information that Angular uses to understand how a component should work.

This metadata is defined inside the `@Component` **decorator**.

Example of Component Metadata

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
})
export class UserComponent {
  name = "Lakshmi";
}
```

Here, everything inside `@Component({ ... })` is called **component metadata**.

Example metadata:

```
{
  selector: 'app-user',
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
}
```

Common Component Metadata Properties

1. selector

Defines the **HTML tag used to display the component**.

Example:

```
selector: 'app-user'
```

Usage in HTML:

```
<app-user></app-user>
```

Angular replaces this tag with the component UI.

2. template

Defines the **HTML template directly inside the component**.

Example:

```
template: `<h1>Hello {{name}}</h1>`
```

3. templateUrl

Specifies the **path to an external HTML template file**.

Example:

```
templateUrl: './user.component.html'
```

4. styleUrls

Specifies CSS files for styling the component.

Example:

```
styleUrls: ['./user.component.css']
```

5. providers

Defines services available for the component.

Example:

```
providers: [UserService]
```

How Metadata Works

Flow:

```
@Component Metadata
    ↓
Angular reads metadata
    ↓
Creates component
    ↓
Renders template in browser
```

Without metadata, Angular **cannot understand how to render or use the component.**

Simple Example

Component:

```
@Component({
  selector: 'app-welcome',
  template: `<h2>Welcome to Angular</h2>`
})
export class WelcomeComponent {}
```

Metadata tells Angular:

- selector → how to use the component
- template → what UI to display

Short Interview Answer

You can answer like this:

Component metadata in Angular refers to the configuration information defined inside the `@Component` decorator. It tells Angular how to create and use the component by specifying details such as selector, template, styles, and providers.

What are directives in Angular?

In **Angular**, a **directive** is a feature that lets you **change the behavior or appearance of elements in the DOM (HTML page)**.

In simple words:

A directive is a class in Angular that adds extra behavior to HTML elements.

It tells Angular **how to modify the DOM** (show, hide, style, or add functionality to elements).

1. Simple Example (Without Directive)

Normal HTML:

```
<p>Hello</p>
```

This just displays text.

But Angular allows us to **attach behavior to HTML** using directives.

Example:

```
<p *ngIf="isLoggedIn">Welcome User</p>
```

Here:

- `*ngIf` is a **directive**
- It tells Angular **show this element only if condition is true**

If `isLoggedIn` = `true` → element appears

If `isLoggedIn` = `false` → element removed from DOM

2. Why Angular Uses Directives

Directives help developers **control the DOM declaratively**.

Declaratively means:

You **describe what should happen**, instead of writing manual JavaScript code.

Example:

Instead of writing JS like:

```
if(isLoggedIn){  
  
document.getElementById("msg").style.display="block";  
}
```

Angular allows:

```
<p *ngIf="isLoggedIn">Welcome</p>
```

Much **cleaner and easier to maintain**.

3. Types of Directives in Angular

Angular has **3 types of directives**.

1. Component Directive

A **Component** is actually a directive with a template.

Example:


```
@Component({
  selector: 'app-user',
  template: `<h1>User Component</h1>`
})
export class UserComponent {}
```

Usage:

```
<app-user></app-user>
```

Here Angular **creates a custom HTML tag**.

2. Structural Directives

Structural directives **change the structure of the DOM**.

They **add or remove elements**.

Examples:

- *ngIf
- *ngFor
- *ngSwitch

Example:

```
<ul>
  <li *ngFor="let item of items">{{item}}</li>
</ul>
```

If items = ['A', 'B', 'C']

Output:

A
B
C

Angular **creates multiple elements automatically.**

3. Attribute Directives

Attribute directives **change the appearance or behavior of elements.**

They **do NOT add/remove elements.**

Examples:

- ngClass
- ngStyle
- ngModel

Example:

```
<p [ngStyle]="{'color':'red'}">Hello</p>
```

This changes the **style of the element.**

4. Example of Custom Directive

You can create your own directive.

Example: Highlight text.

```
import { Directive, ElementRef } from
 '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor =
    'yellow';
  }

}
```

Usage:

```
<p appHighlight>Highlighted text</p>
```

Output:

The paragraph background becomes **yellow**.

5. What is ElementRef?

In this code:

```
constructor(private el: ElementRef)
```

ElementRef gives **direct access to the DOM element**.

Example:

```
this.el.nativeElement.style.color = 'blue';
```

This changes the **color of the HTML element**.

6. Quick Summary

Concept	Meaning
Directive	Class that adds behavior to HTML elements
Purpose	Modify DOM
Structural Directive	Changes DOM structure (*ngIf, *ngFor)
Attribute Directive	Changes appearance (ngStyle, ngClass)
Component	Directive with template

7. One-Line Interview Answer

Directives in Angular are classes that allow us to extend the behavior of HTML elements by modifying the DOM. Angular provides three types of directives: components, structural directives, and attribute directives.

What are structural directives?

Structural Directives in Angular

In **Angular**, **Structural Directives** are directives that **change the structure of the DOM** by **adding, removing, or replacing HTML elements**.

In simple words:

Structural directives control whether an element exists in the DOM or not.

They decide **what should appear on the page**.

1. Simple Example

```
<p *ngIf="isLoggedIn">Welcome User</p>
```

Here:

- `*ngIf` is a **structural directive**
- If `isLoggedIn` = `true` → paragraph is shown
- If `isLoggedIn` = `false` → paragraph is **removed from DOM**

So Angular **physically adds or removes the element**.

2. Why They Are Called “Structural”

Because they **change the structure of the HTML page (DOM)**.

Example DOM:

Before Angular processes it:

```
<p>Welcome User</p>
```

If condition is false, Angular **removes the element completely**.

Final DOM:

```
<!-- element not present -->
```

So the **structure of the page changes**.

3. Common Structural Directives

Angular provides several built-in structural directives.

1. *ngIf

Used to **show or hide elements based on a condition**.

Example:

```
<p *ngIf="isAdmin">Admin Panel</p>
```

If `isAdmin = true` → element appears

If `false` → element removed.

2. *ngFor

Used to **repeat elements in a loop**.

Example:

```
<ul>
  <li *ngFor="let user of users">
    {{user}}
  </li>
</ul>
```

If:

```
users = ['Ram', 'Ravi', 'John'];
```

Angular generates:

```
<li>Ram</li>
<li>Ravi</li>
<li>John</li>
```

Angular **creates multiple elements dynamically.**

3. *ngSwitch

Used for **multiple conditional rendering.**

Example:

```
<div [ngSwitch]="role">

  <p *ngSwitchCase="'admin'">Admin View</p>
  <p *ngSwitchCase="'user'">User View</p>
  <p *ngSwitchDefault>Guest View</p>
```

```
</div>
```

Angular shows the element depending on the value of role.

4. Why * Is Used in Structural Directives

The * is shorthand syntax.

Example:

```
<p *ngIf="isLoggedIn">Welcome</p>
```

Angular internally converts it into:

```
<ng-template [ngIf]="isLoggedIn">  
  <p>Welcome</p>  
</ng-template>
```

Angular wraps the element inside ng-template, which controls whether it is rendered.

5. Important Rule

You **cannot use two structural directives on the same element.**

✗ Wrong:

```
<li *ngIf="isLoggedIn" *ngFor="let user of  
users"></li>
```


✓ Correct:

```
<div *ngIf="isLoggedIn">
  <li *ngFor="let user of users"></li>
</div>
```

6. Structural vs Attribute Directives

Feature	Structural Directive	Attribute Directive
Purpose	Changes DOM structure	Changes style or behavior
DOM elements	Added or removed	Not removed
Example	*ngIf, *ngFor	ngStyle, ngClass

7. Real-World Example

Example: Display products.

```
<div *ngFor="let product of products">
  <p>{{product.name}}</p>
</div>
```

Angular dynamically **creates product elements** depending on the data.

8. Interview Definition

Structural directives are Angular directives that change the DOM layout by adding or removing elements from the DOM. Examples include `*ngIf`, `*ngFor`, and `*ngSwitch`.

What are attribute directives?

Attribute Directives in Angular

In **Angular**, **Attribute Directives** are directives that **change the appearance or behavior of an existing HTML element**.

They **do NOT** add or remove elements from the DOM.

Attribute directives modify the style, class, or behavior of an element.

1. Simple Example

```
<p [ngStyle]="{'color':'red'}">Hello</p>
```

Here:

- `ngStyle` is an **attribute directive**
- It **changes the style of the element**

Output:

The text **Hello** appears in **red color**.

The element still exists in the DOM — only its **appearance changes**.

2. Why They Are Called Attribute Directives

Because they are applied like **HTML attributes**.

Example:

```
<p ngClass="highlight">Hello</p>
```

Here ngClass behaves like an **HTML attribute** but adds Angular behavior.

3. Common Attribute Directives in Angular

1. ngClass

Used to **add or remove CSS classes dynamically**.

Example:

```
<p [ngClass]="{'active': isActive}">  
  User Status  
</p>
```

If isActive = true

Output:

```
<p class="active">User Status</p>
```

2. ngStyle

Used to **apply styles dynamically**.

Example:

```
<p [ngStyle]="{'font-size':'20px', 'color':'blue'}">  
  Angular  
</p>
```

This changes:

- font size
- color

3. ngModel

Used for **two-way data binding in forms**.

Example:

```
<input [(ngModel)]="username">
```

If the user types something, Angular **updates the variable automatically**.

4. Example of Custom Attribute Directive

You can create your own directive.

Example: Highlight text.

Step 1: Create Directive

```
import { Directive, ElementRef } from  
'@angular/core';
```

```
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor =
'yellow';
  }

}
```

Step 2: Use Directive

```
<p appHighlight>Highlighted text</p>
```

Output:

Text background becomes **yellow**.

5. How It Works Internally

Angular attaches the directive to the element and changes its properties.

Example:

Original HTML

```
<p>Hello</p>
```

After directive

```
<p style="background-color:yellow">Hello</p>
```

The element remains in the DOM — only its **style changes**.

6. Attribute vs Structural Directives

Feature	Structural Directive	Attribute Directive
Purpose	Changes DOM structure	Changes appearance or behavior
DOM elements	Added or removed	Not removed
Symbol	* used	No *
Examples	*ngIf, *ngFor	ngClass, ngStyle, ngModel

7. Easy Way to Remember

Structural Directive

→ Changes **structure of HTML**

Example:

*ngIf
*ngFor

Attribute Directive

→ Changes **style or behavior**

Example:

ngClass
ngStyle
ngModel

8. Interview Definition

Attribute directives are Angular directives that change the appearance or behavior of an existing DOM element without modifying its structure. Examples include ngClass, ngStyle, and ngModel.

What is ngClass in Angular?

ngClass is an **attribute directive** used to **add or remove CSS classes dynamically** based on a condition.

In simple words:

ngClass allows Angular to apply CSS classes to HTML elements depending on a condition.

1. First Understand Normal CSS Class

Normal HTML:

```
<p class="highlight">Hello</p>
```

CSS:

```
.highlight {  
  color: red;  
  font-weight: bold;  
}
```

Output:

The text **Hello** becomes **red and bold**.

But here the class is **fixed**.

2. Problem Without ngClass

Suppose we want:

- If user is **active** → **green text**
- If user is **inactive** → **red text**

Without Angular, we would write complex JavaScript.

Angular solves this using ngClass.

3. Basic Example of ngClass

HTML

```
<p [ngClass]="{'active': isActive}">  
  User Status  
</p>
```

CSS

```
.active {  
  color: green;  
}
```

TypeScript

```
isActive = true;
```

What Angular Does

If `isActive = true`

```
<p class="active">User Status</p>
```

If `isActive = false`

```
<p>User Status</p>
```

So Angular **adds or removes the class automatically**.

4. Real Life Example

Suppose we want to show:

- **Online** → green
- **Offline** → red

CSS

```
.online {  
  color: green;  
}  
  
.offline {  
  color: red;  
}
```

HTML

```
<p [ngClass]="isOnline ? 'online' : 'offline'">  
  User Status  
</p>
```

TypeScript

```
isOnline = true;
```

Output

If true

User Status (green)

If false

User Status (red)

5. Multiple Classes Example

ngClass can apply **multiple classes**.

HTML

```
<p [ngClass]="{  
  'active': isActive,  
  'bold': isImportant  
}">  
  Angular Example  
</p>
```

TypeScript

```
isActive = true;  
isImportant = true;
```

Angular adds both classes:

```
<p class="active bold">Angular Example</p>
```

6. Very Simple Visualization

Normal HTML

```
<p>Hello</p>
```

With ngClass

Angular checks condition

↓

If true → add class

If false → remove class

Final DOM

```
<p class="active">Hello</p>
```

or

```
<p>Hello</p>
```

7. Difference Between class and ngClass

Feature	class	ngClass
Static or dynamic	Static	Dynamic
Condition support	No	Yes

Angular feature	No	Yes
-----------------	----	-----

Example:

Static

```
<p class="red">Hello</p>
```

Dynamic

```
<p [ngClass]="{'red': isError}">Hello</p>
```

8. One-Line Interview Answer

ngClass is an Angular attribute directive used to dynamically add or remove CSS classes from an HTML element based on conditions.

What is the difference between *ngIf* and *ngFor*?

In Angular, **ngIf* and **ngFor* are both **structural directives**, but they are used for **different purposes**.

Structural directives change the **structure of the DOM** by **adding or removing elements**.

Let's understand the difference clearly.

1. *ngIf

*ngIf is used to **conditionally display or hide an element**.

It **adds or removes an element from the DOM based on a condition**.

Example

```
<p *ngIf="isLoggedIn">Welcome User</p>
```

TypeScript

```
isLoggedIn = true;
```

What happens

```
isLoggedIn = true
```

```
<p>Welcome User</p>
```

```
isLoggedIn = false
```

The element **does not exist in the DOM**.

```
<!-- removed from DOM -->
```

So *ngIf is used for **conditional rendering**.

2. *ngFor

*ngFor is used to **repeat an element multiple times** based on a collection (array or list).

Example

```
<ul>
  <li *ngFor="let user of users">
    {{user}}
  </li>
</ul>
```

TypeScript

```
users = ['Ram', 'Ravi', 'John'];
```

What Angular generates

```
<li>Ram</li>
<li>Ravi</li>
<li>John</li>
```

So *ngFor **creates multiple DOM elements dynamically**.

3. Key Difference

Feature	*ngIf	*ngFor
Purpose	Conditional rendering	Looping through data
Works with	Boolean condition	Arrays or lists
DOM behavior	Adds or removes element	Creates multiple elements
Example	Show login message	Display list of users

4. Simple Real-World Example

*ngIf Example

Show message only when user is logged in.

```
<p *ngIf="isLoggedIn">Welcome Back!</p>
```

*ngFor Example

Display list of products.

```
<div *ngFor="let product of products">
  {{product.name}}
</div>
```


Angular creates **multiple product elements automatically**.

5. Important Rule

You **cannot use two structural directives on the same element**.

 Wrong

```
<li *ngIf="showUsers" *ngFor="let user of users"></li>
```

✓ Correct

```
<div *ngIf="showUsers">  
  <li *ngFor="let user of users"></li>  
</div>
```

6. Easy Way to Remember

*ngIf → IF condition (show or hide)

*ngFor → FOR loop (repeat elements)

7. Interview Answer (Short)

***ngIf is used to conditionally add or remove elements from the DOM, while *ngFor is used to iterate over a collection and generate multiple DOM elements dynamically.**

What is the difference between component and directive?

In Angular, **Component** and **Directive** are closely related concepts.

In fact:

A component is a special type of directive that has a template (HTML view).

Both are used to **extend the behavior of HTML**, but they serve different purposes.

Let's understand this clearly.

1. What is a Component?

A **Component** controls a **part of the UI (view)** in an Angular application.

It contains:

- **HTML template (UI)**
- **TypeScript class (logic)**
- **CSS styles**

So a component defines **how a part of the screen looks and behaves**.

Example

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `<h2>User Component</h2>`
})
export class UserComponent { }
```

Usage

```
<app-user></app-user>
```

Angular creates a **new custom HTML element**.

Output:

User Component

So a **component builds UI blocks**.

2. What is a Directive?

A **Directive** is used to **change the behavior or appearance of an existing HTML element**.

It **does not create its own UI**.

Example directive:

```
import { Directive, ElementRef } from
 '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor =
    'yellow';
  }

}
```

Usage

```
<p appHighlight>Angular Directive</p>
```

Output:

The paragraph background becomes **yellow**.

Here the directive **modifies an existing element**.

3. Key Differences

Feature	Component	Directive
---------	-----------	-----------

Purpose	Create UI (view)	Modify behavior of existing elements
Template	Has HTML template	No template
Usage	Used as custom HTML tag	Used as attribute
Decorator	@Component	@Directive
Example	<app-user>	appHighlight

4. Simple Visualization

Component

Creates **new UI block**

```
<app-header></app-header>
<app-login></app-login>
<app-footer></app-footer>
```

Directive

Modifies existing HTML

```
<p appHighlight>Text</p>
```

5. Important Angular Concept

Under the hood:

Component = Directive + Template

All components are **directives**, but not all directives are **components**.

6. Real-World Analogy

Think of a **house**.

Component → **Room**

- Bedroom
- Kitchen
- Hall

Directive → **Decoration or behavior**

- Paint color
- Lighting
- Curtain style

Directives **enhance existing elements**, while components **create UI sections**.

7. Interview Answer (Short)

A component is a special type of directive that has a template and is used to build UI parts of an Angular application, whereas a

directive is used to modify the behavior or appearance of existing DOM elements without creating a view.

What are Angular lifecycle hooks?

Angular lifecycle hooks are **special methods** that Angular automatically calls at different stages of a component's life — from creation → rendering → updates → destruction.

Think of a component like a human life 👉

Born → grows → changes → dies

Similarly, Angular components also go through stages — and **lifecycle hooks let you control what happens at each stage.**

Complete Angular Lifecycle Flow (Simple Understanding)

1. Component is **created**
2. Inputs (data from parent) are set
3. Component is **initialized**
4. View (HTML) is rendered
5. Change detection runs multiple times
6. Component is **destroyed**

Why Lifecycle Hooks?

You use them to:

- Initialize data
- Fetch API data

- Detect changes
- Access DOM
- Cleanup memory (unsubscribe, etc.)

`ngOnChanges()` is one of the most important **Angular lifecycle hooks**, especially when dealing with **parent → child communication**.

I'll explain this in a **clear, step-by-step way with code**, so you can fully understand it.

What is `ngOnChanges()`?

`ngOnChanges()` is a lifecycle hook that:

- 👉 Runs **whenever an @Input property value changes**
- 👉 Used to **detect changes coming from parent component**

When does it trigger?

It runs in two cases:

1. **First time** → when child receives data from parent
2. **Later** → whenever parent updates that data

How Parent & Child are connected?

In Angular, we don't use extends like Java.

Instead, we use:

- 👉 `@Input()` → to receive data from parent

Real Example (VERY IMPORTANT)

Parent Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-parent',
  template: `
    <h2>Parent Component</h2>
    <input [(ngModel)]="message" placeholder="Enter
message">

    <app-child [message]="message"></app-child>
  `
})
export class ParentComponent {
  message: string = 'Hello Child';
}
```

Child Component

```
import { Component, Input, OnChanges, SimpleChanges }
from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <h3>Child Component</h3>
    <p>Message: {{ message }}</p>
  `
})
```

```
})  
export class ChildComponent implements OnChanges {  
  @Input() message!: string;  
  
  ngOnChanges(changes: SimpleChanges) {  
    console.log('Changes detected:', changes);  
  
    if (changes['message']) {  
      console.log('Previous:',  
changes['message'].previousValue);  
      console.log('Current:',  
changes['message'].currentValue);  
    }  
  }  
}
```

What is @Input()?

```
@Input() message!: string;
```

👉 Means:

"Hey Angular, this variable will get value **from parent component**"

So:

```
<app-child [message]="message"></app-child>
```

👉 Parent → Child data binding

How ngOnChanges() works internally

When parent updates:

```
message = "New Value";
```

Angular will:

1. Detect change
2. Update child @Input()
3. Call ngOnChanges()

What is SimpleChanges?

Inside ngOnChanges():

```
ngOnChanges(changes: SimpleChanges)
```

👉 changes is an object like:

```
{  
  message: {  
    previousValue: 'Hello Child',  
    currentValue: 'New Value',  
    firstChange: false  
  }  
}
```

Key Properties

Property	Meaning
previousValue	Old value
currentValue	New value
firstChange	true if first time

Important Notes

- ✓ Only works with `@Input()`
- ✓ Not triggered for normal variables
- ✓ Called **before ngOnInit()** (first time)
- ✓ Called multiple times when value changes

Order of Execution

When component loads:

```
constructor()  
↓  
ngOnChanges()   ✓ (first time)  
↓  
ngOnInit()
```

Common Use Cases

👉 When you want to:

- React to input changes
- Call API when input changes
- Validate new data
- Update derived values

Simple Real-Life Analogy

Think like:

👤 Parent = Boss

👤 Child = Employee

Boss gives task:

👉 First time → `ngOnChanges()` runs

👉 Every update → `ngOnChanges()` runs again

Interview Answer (Short)

👉 "`ngOnChanges()` is a lifecycle hook that is triggered whenever an `@Input()` property changes. It receives `SimpleChanges` object which contains previous and current values."

What is `ngOnInit()`?

`ngOnInit()` is a lifecycle hook that:

👉 Runs **only once** when the component is initialized

👉 Used for **initial setup logic**

When does it run?

Execution order:

```
constructor()  
↓  
ngOnChanges() (if @Input present)  
↓  
ngOnInit()    (runs ONCE)
```

👉 It runs **after Angular sets all input values**

Why not use constructor?

Good question (very important for interviews)

✗ Constructor

- Used for **dependency injection**
- Component not fully ready

ngOnInit

- Component is **fully initialized**
- Safe to write logic

Real Example

✓ Component Code

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-demo',
  template: `
    <h2>User Info</h2>
    <p>Name: {{ name }}</p>
  `
})
export class DemoComponent implements OnInit {

  name: string = '';

  constructor() {
    console.log('Constructor called');
  }

  ngOnInit() {
    console.log('ngOnInit called');

    // initialization logic
    this.name = 'Lakshmi';
  }
}
```

What happens internally?

When Angular loads component:

1. Creates component → constructor()
2. Sets input values (if any)
3. Calls `ngOnInit()`

👉 That's why initialization logic should go here

Real Use Cases

Use `ngOnInit()` for:

1. API Calls

```
ngOnInit() {  
  this.loadUsers();  
}  
  
loadUsers() {  
  // call backend API  
}
```

2. Initial Data Setup

```
ngOnInit() {  
  this.isLoggedIn = true;  
}
```


3. Subscriptions

```
ngOnInit() {  
  this.service.getData().subscribe(data => {  
    this.data = data;  
  });  
}
```

Important Notes

- ✓ Runs **only once**
- ✓ Called after `@Input()` values are set
- ✓ NOT called again on changes
- ✓ Best place for initialization logic

Difference: `ngOnInit` vs `ngOnChanges`

Feature	<code>ngOnInit</code>	<code>ngOnChanges</code>
Runs how many times	Once	Multiple times
Trigger	Component init	@Input change
Use case	Initial setup	React to changes

Common Mistake

- ❌ Doing API calls in constructor
- ❌ Expecting `ngOnInit` to run multiple times
- 👉 It will NOT re-run

Interview Answer (Short)

👉 "`ngOnInit()` is a lifecycle hook that is called once after Angular initializes the component and sets its input properties. It is mainly used for initialization logic like API calls and data setup."

Simple Analogy

Think like:

🏠 You enter a new house

👉 `ngOnInit()` = setting up furniture (done once)

Later changes → NOT handled here

Note:

👉 **Short answer:**

❌ `ngOnChanges()` does NOT run on every change detection

✅ It runs **only when an `@Input()` value changes (from parent)**

What is ngDoCheck()?

ngDoCheck() is a lifecycle hook that:

- Runs **on every change detection cycle**
- Allows you to implement **custom change detection logic**

👉 It gives you **full control** to detect changes manually.

When does ngDoCheck() run?

Angular runs change detection in many situations:

- User events (click, input typing)
- API responses
- setTimeout / setInterval
- Promise resolution
- Parent component updates

👉 During **each of these**, ngDoCheck() is triggered.

Execution Flow

Event happens



Change Detection runs



ngDoCheck() is called

Basic Example

```
import { Component, DoCheck } from '@angular/core';

@Component({
  selector: 'app-demo',
  template: `
    <button (click)="increment()">Click Me</button>
  `
})
export class DemoComponent implements DoCheck {

  count = 0;

  ngDoCheck() {
    console.log('ngDoCheck triggered');
  }

  increment() {
    this.count++;
  }
}
```

👉 Every button click → `ngDoCheck()` runs

Why do we need `ngDoCheck()`?

Angular's default change detection works well for:

- Primitive values (string, number, boolean)

But it may fail for:

- Objects
- Arrays (when reference does not change)

Problem Example

```
@Input() user: any;

ngOnChanges() {
  console.log('ngOnChanges triggered');
}
```

Parent updates:

```
this.user.name = "Lakshmi"; // only property changed
```

👉 Reference not changed

👉 ngOnChanges() ❌ NOT triggered

Solution Using ngDoCheck()

```
import { Component, Input, DoCheck } from
 '@angular/core';

@Component({
  selector: 'app-child',
  template: `<p>{{ user.name }}</p>`
})
export class ChildComponent implements DoCheck {

  @Input() user: any;
```

```

previousName = '';

ngDoCheck() {
  if (this.user && this.user.name !==
this.previousName) {
    console.log('Detected change manually');

    this.previousName = this.user.name;
  }
}
}

```

👉 Now changes are detected manually

Key Differences: ngOnChanges vs ngDoCheck

Feature	ngOnChanges	ngDoCheck
Trigger	@Input changes	Every change detection
Frequency	Limited	Very frequent
Purpose	React to input changes	Custom detection

Important Notes

- Runs **very frequently**
- Can affect performance if used incorrectly
- Should contain **lightweight logic only**

When to Use `ngDoCheck()`

Use it when:

- Angular fails to detect changes
- Working with deep object mutations
- Need custom comparison logic

When **NOT** to Use

Avoid when:

- Normal `@Input()` change detection is enough
- You can use immutable updates instead

Best Practice (Recommended Approach)

Instead of mutating objects:

```
this.user.name = "Lakshmi"; ❌
```

Use immutable update:

```
this.user = { ...this.user, name: "Lakshmi" }; ✅
```

👉 This triggers `ngOnChanges()`

👉 No need for `ngDoCheck()`

Execution Order with Other Hooks

```
constructor()  
↓  
ngOnChanges()  
↓  
ngOnInit()  
↓  
ngDoCheck() (runs many times)
```

Real-World Insight

👉 Think of `ngDoCheck()` as:

- A **manual inspector**
- Runs repeatedly
- Checks for things Angular might miss

Interview Answer

`ngDoCheck()` is a lifecycle hook that is called during every change detection cycle. It is used to implement custom change detection logic when Angular's default mechanism is not sufficient, especially for detecting changes in objects and arrays.

Final Takeaway

- `ngDoCheck()` is powerful but **should be used carefully**
- Prefer Angular's default change detection whenever possible
- Use it only when absolutely necessary

`ngDoCheck()`:

- `ngDoCheck()` is used for **custom change detection logic**
- Angular's default change detection may not detect:
 - object property changes
 - array mutations (same reference)
- In such cases, we use `ngDoCheck()` to **manually detect changes**
- It also runs during every change detection cycle, so we can **add additional checks if needed**

`ngOnChanges()`:

- `ngOnChanges()` runs **only when `@Input()` properties change**
- Used mainly for **parent → child communication**

Angular detects:

- ✓ Primitive changes (string, number)
- ✓ Reference changes (new object/array)

✗ But may NOT detect:

- Deep object changes (`user.name` = 'x')

- Array mutations (push, pop)

Final Correct Summary (You can use this 👍)



- Angular has a **default change detection mechanism** that automatically updates the UI
- `ngOnChanges()` is triggered **only when** `@Input()` **values change**
- `ngDoCheck()` is triggered **on every change detection cycle** and is used for **custom change detection logic**
- `ngDoCheck()` is useful when Angular's default change detection **fails to detect changes in objects or arrays**

Angular detects → `ngOnChanges` reacts → `ngDoCheck` inspects deeply

ngAfterContentInit

🔗 **What is `ngAfterContentInit`?**

`ngAfterContentInit()` is a **lifecycle hook** in Angular that runs:

👉 **After Angular projects external content (from parent) into the child component**

In simple words:

When **parent HTML** is inserted into child using `<ng-content>`, this hook is called **once after that content is initialized**.

First understand: Content Projection

👉 What is "content"?

Content = **HTML/data passed from parent to child component**

Example:

```
<app-child>
  <p>This is coming from parent</p>
</app-child>
```

What is "projected content"?

Projected content =

Parent content that gets inserted inside child's `<ng-content>`

Flow of Execution

1. Parent sends HTML inside `<app-child>`
2. Child receives it via `<ng-content>`
3. Angular inserts that content into child
4.  `ngAfterContentInit()` gets triggered

Important Point

- ✓ Runs **only once** (not multiple times)
- ✓ Called **after content projection is done**
- ✓ Used when you want to **access or modify projected content**

Complete Example (Parent + Child)

Child Component

```
import { Component, AfterContentInit, ContentChild,
ElementRef } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <div style="border: 2px solid blue; padding:
10px;">
      <h3>Child Component</h3>
      <ng-content></ng-content>
    </div>
  `
})
export class ChildComponent implements
AfterContentInit {

  @ContentChild('projectedPara') para!: ElementRef;

  ngAfterContentInit() {
    console.log('ngAfterContentInit called');

    // Access projected content
    console.log(this.para.nativeElement.innerText);

    // Modify projected content
    this.para.nativeElement.style.color = 'red';
  }
}
```

```
}
```

Parent Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <h2>Parent Component</h2>

    <app-child>
      <p #projectedPara>This is projected from
parent</p>
    </app-child>
  `
})
export class AppComponent {}
```

What is happening here?

1. Parent sends:

```
<p #projectedPara>This is projected from parent</p>
```

2. Child receives it via:

```
<ng-content></ng-content>
```

3. Angular inserts content

4. Then:

```
ngAfterContentInit()
```

runs

5. Using:

```
@ContentChild('projectedPara')
```

child accesses parent content

Difference from `ngAfterViewInit`

Hook	Works On
<code>ngAfterContentInit</code>	Parent → Child projected content
<code>ngAfterViewInit</code>	Child's own HTML (template)

Real Use Cases

- ✓ Access projected elements
- ✓ Modify styles/content of projected data
- ✓ Perform logic after parent content is available
- ✓ Useful in reusable UI components (cards, wrappers, layouts)

Short Memory Trick

◆ **Content = Parent data**

◆ **View = Child template**

So:

👉 `AfterContentInit` → Parent content ready

👉 `AfterViewInit` → Child view ready

ngAfterContentChecked

🔗 **What is `ngAfterContentChecked`?**

`ngAfterContentChecked()` is a lifecycle hook that runs:

👉 **Every time Angular checks projected content (parent → child)**

Simple Understanding

If `ngAfterContentInit` = runs **once after content comes**

👉 Then

`ngAfterContentChecked` = **runs every time that content is checked/updated**

Execution Flow

Whenever Angular change detection runs:

1. Parent updates data

2. That data is projected into child (<ng-content>)
3. Angular checks the content
4.  ngAfterContentChecked() is triggered

Important Points

- ✓ Runs **multiple times** (very frequently)
- ✓ Runs **after every change detection cycle**
- ✓ Can impact performance if heavy logic is written
- ✓ Used to **track updates in projected content**

Complete Example (Parent + Child)

Child Component

```
import { Component, AfterContentInit,
AfterContentChecked, ContentChild, ElementRef } from
'@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <div style="border: 2px solid green; padding:
10px;">
      <h3>Child Component</h3>
      <ng-content></ng-content>
    </div>
  `
})
export class ChildComponent implements
```



```

AfterContentInit, AfterContentChecked {

  @ContentChild('projectedPara') para!: ElementRef;

  ngAfterContentInit() {
    console.log('ngAfterContentInit called');
  }

  ngAfterContentChecked() {
    console.log('ngAfterContentChecked called');

    if (this.para) {
      console.log('Current content:',
this.para.nativeElement.innerText);
    }
  }
}

```

Parent Component

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <h2>Parent Component</h2>

    <button (click)="changeText()">Change
Content</button>

    <app-child>

```

```
    <p #projectedPara>{{ message }}</p>
  </app-child>
  ,
  ))
export class AppComponent {
  message = 'Initial Content';

  changeText() {
    this.message = 'Updated Content ' + new
Date().toLocaleTimeString();
  }
}
```

What Happens Step-by-Step?

1. Initial load

👉 `ngAfterContentInit()` runs once

👉 `ngAfterContentChecked()` runs

2. Click button

👉 Parent updates message

👉 Content inside `<ng-content>` changes

👉 Angular runs change detection

👉 ✅ `ngAfterContentChecked()` runs again

Difference: Init vs Checked

Hook	When it runs	Frequency
ngAfterContentInit	After first content projection	 Once
ngAfterContentChecked	After every content check	 Multiple times

Real Use Cases

- ✓ Detect changes in projected content
- ✓ Debug content updates
- ✓ Sync child logic with parent-provided content
- ✓ Validate projected data

Performance Warning (Very Important)

✗ Don't write heavy logic like:

```
ngAfterContentChecked() {  
  // API calls ✗  
  // Complex calculations ✗  
}
```

Because it runs **many times**

Memory Trick

◆ **Init = First time only**

◆ **Checked = Every time Angular checks**

Final Understanding

- `ngAfterContentInit` → “Content came first time”
- `ngAfterContentChecked` → “Content checked again and again”

ngAfterViewInit():

`ngAfterViewInit()` is one of Angular’s **lifecycle hooks**.

👉 It is called **once**, after Angular **completes loading the component’s view (HTML)** and **all child components inside it**.

Simple Meaning

Think like this:

- `ngOnInit()` → Component created (data ready)
- `ngAfterViewInit()` → **HTML + child components fully rendered in DOM**

Execution Flow (Important)

Angular lifecycle order (simplified):

```
constructor()  
↓  
ngOnChanges()  
↓  
ngOnInit()  
↓  
ngAfterContentInit()  
↓  
ngAfterContentChecked()  
↓  
ngAfterViewInit()  (our focus)  
↓  
ngAfterViewChecked()
```

Why ngAfterViewInit() is Needed?

Because in ngOnInit():

 DOM (HTML elements) may NOT be fully ready

 Child components may NOT be available

But in ngAfterViewInit():

 HTML elements are available

 Child components are available

 Safe to access DOM and @ViewChild

What are "Child Components"?

Example:

Child Component

```
@Component({
  selector: 'app-child',
  template: `<p>I am Child</p>`
})
export class ChildComponent {}
```

Parent Component HTML

```
<h1>Parent Component</h1>
<app-child></app-child>  <!-- Child component -->
```

👉 Here:

- ParentComponent → parent
- <app-child> → **child component**

Example with @ViewChild

Parent Component

```
import { Component, AfterViewInit, ViewChild,
  ElementRef } from '@angular/core';

@Component({
  selector: 'app-parent',
```

```
template: `
  <h1 #title>Parent Component</h1>
  <app-child></app-child>
`
})
export class ParentComponent implements AfterViewInit {

  @ViewChild('title') title!: ElementRef;

  ngAfterViewInit() {
    console.log(this.title.nativeElement.innerText);
  }
}
```

What is happening here?

- #title → reference to HTML element
- @ViewChild('title') → gets that element
- But this works **only after view is initialized**

 That's why we use:

ngAfterViewInit()

Important Point

If you try this in ngOnInit():

```
ngOnInit() {  
  console.log(this.title); // ❌ undefined  
}
```

👉 Because DOM is not ready yet.

Real Use Cases

Use `ngAfterViewInit()` when you need:

Access DOM elements

```
this.title.nativeElement.style.color = 'red';
```

Work with child components

```
@ViewChild(ChildComponent) childComp!:  
ChildComponent;
```

Integrate third-party libraries

Example:

- Charts (Chart.js)
- DOM plugins

Key Difference (Very Important)

Hook	What is ready?
------	----------------

ngOnInit()	Only component data
ngAfterViewInit()	HTML + child components

Easy Analogy

- ngOnInit() → House built 🏠
- ngAfterViewInit() → Furniture placed + ready to use 🚗

Final Summary

- Runs **once**
- Executes **after HTML & child components are rendered**
- Used for:
 - DOM access
 - @ViewChild
 - UI manipulation

ngAfterViewChecked()

ngAfterViewChecked() is another **Angular lifecycle hook**.

👉 It is called **after Angular checks the component's view (HTML) and child components**

👉 And it runs **MANY times**, not just once

Simple Meaning

Think like this:

- `ngAfterViewInit()` → runs **once** after view is created
- `ngAfterViewChecked()` → runs **every time Angular checks the view**

When exactly it runs?

Whenever Angular does **change detection**, this hook runs:

✓ Data changes

✓ Button click

✓ Input change

✓ API response

✓ Any UI update

👉 After each of these → Angular checks the view → then calls:

`ngAfterViewChecked()`

Execution Order (Important)

`ngAfterViewInit()`  (runs once)

↓

`ngAfterViewChecked()`  (runs multiple times)

Example

```
import { Component, AfterViewInit, AfterViewChecked }  
from '@angular/core';
```

```
@Component({
  selector: 'app-demo',
  template: `
    <h1>{{count}}</h1>
    <button (click)="increase()">Increase</button>
  `
})
export class DemoComponent implements AfterViewInit,
AfterViewChecked {

  count = 0;

  ngAfterViewInit() {
    console.log('AfterViewInit - runs once');
  }

  ngAfterViewChecked() {
    console.log('AfterViewChecked - runs again &
again');
  }

  increase() {
    this.count++;
  }
}
```

What happens here?

1. Component loads

👉 `ngAfterViewInit()` runs once

👉 `ngAfterViewChecked()` runs

2. Click button

👉 count changes

👉 Angular runs change detection

👉 `ngAfterViewChecked()` runs AGAIN

VERY IMPORTANT (Interview Point)

! This hook runs **very frequently**

So:

🚫 Don't write heavy logic

🚫 Don't call APIs

🚫 Don't update values continuously

👉 Otherwise → performance issues or infinite loops

Dangerous Example (Avoid This)

```
ngAfterViewChecked() {  
  this.count++; // ❌ BAD  
}
```

👉 This causes:

- change → detection → hook → change → detection → loop 🌀

Correct Use Cases

Use it when you want to:

- ✓ Track view updates
- ✓ Debug UI changes
- ✓ Run small checks after view update

Example:

```
ngAfterViewChecked() {  
  console.log('View updated');  
}
```

Difference from `ngAfterViewInit()`

Feature	<code>ngAfterViewInit</code>	<code>ngAfterViewChecked</code>
Runs	Once	Multiple times
When	After first render	After every change detection
Use	Initial DOM access	Monitor updates

Easy Analogy

- `ngAfterViewInit()` → First time opening a website

- `ngAfterViewChecked()` → Every time you refresh or interact

Final Summary

- Runs **after every view check**
- Called **many times**
- Used for **tracking updates (not heavy logic)**
- Be careful → can affect performance

What is `ngOnDestroy()`?

👉 `ngOnDestroy()` is called **just before Angular destroys the component**.

This happens when:

- You navigate to another page (routing)
- The component is removed using `*ngIf`
- The parent component removes it

Why do we need it?

When a component is destroyed, **Angular does NOT automatically clean everything**.

So we use `ngOnDestroy()` to:

- ❌ Stop subscriptions (Observables)
- ❌ Clear timers (`setInterval`, `setTimeout`)
- ❌ Remove event listeners
- ❌ Avoid memory leaks

Problem without ngOnDestroy()

Example:

- You subscribe to an API or interval
- Then navigate away
- That subscription still runs in background !

👉 This causes:

- Memory leaks
- Unexpected behavior

Example (Real Interview-Level Explanation)

Without Cleanup (Wrong Way)

```
export class DemoComponent implements OnInit {  
  ngOnInit() {  
    setInterval(() => {  
      console.log('Running...');  
    }, 1000);  
  }  
}
```

✗ Even after leaving the page → still runs

With ngOnDestroy() (Correct Way)

```
import { Component, OnInit, OnDestroy } from  
'@angular/core';
```

```

@Component({
  selector: 'app-demo',
  template: `<h2>Demo Component</h2>`
})
export class DemoComponent implements OnInit,
OnDestroy {

  intervalId: any;

  ngOnInit() {
    this.intervalId = setInterval(() => {
      console.log('Running...');
    }, 1000);
  }

  ngOnDestroy() {
    clearInterval(this.intervalId);
    console.log('Component Destroyed');
  }
}

```

Example with Observable (Very Important)

```

import { Component, OnInit, OnDestroy } from
 '@angular/core';
import { interval, Subscription } from 'rxjs';

export class DemoComponent implements OnInit,
OnDestroy {

```



```
sub!: Subscription;

ngOnInit() {
  this.sub = interval(1000).subscribe(val => {
    console.log(val);
  });
}

ngOnDestroy() {
  this.sub.unsubscribe(); //  Important
}
}
```

When exactly it runs?

Lifecycle order (simplified):

```
ngOnInit()
↓
ngAfterViewInit()
↓
... (many changes)
↓
ngOnDestroy()  (last step)
```

Interview Answer (Perfect)

 *"ngOnDestroy is a lifecycle hook that is called just before Angular destroys a component. It is mainly used to clean up*

resources like subscriptions, timers, and event listeners to prevent memory leaks."

What are Angular Lifecycle Hooks?

Answer:

Angular lifecycle hooks are predefined methods that Angular automatically calls at different stages of a component's lifecycle, such as creation, change detection, rendering, and destruction. They allow us to control logic like initialization, detecting input changes, interacting with the DOM, and cleanup.

Explain all lifecycle hooks

Answer:

- **constructor()**

Used for dependency injection. It is not an Angular hook. At this stage, the component is created but inputs and DOM are not ready.

- **ngOnChanges()**

Called whenever input properties change. It is used to detect changes from parent to child components and gives previous and current values.

- **ngOnInit()**

Called once after component initialization. Used for API calls and initializing data. Inputs are available but DOM is not fully ready.

- **ngDoCheck()**

Called during every change detection cycle. Used for custom change detection logic, especially for objects and arrays.

- **ngAfterContentInit()**

Called once after content projection (ng-content) is initialized. Used when working with projected content.

- **ngAfterContentChecked()**

Called after every check of projected content. Runs multiple times.

- **ngAfterViewInit()**

Called once after component view (HTML) and child components are initialized. Used to access DOM and @ViewChild.

- **ngAfterViewChecked()**

Called after every view check. Runs multiple times. Should not contain heavy logic.

- **ngOnDestroy()**

Called before component is destroyed. Used for cleanup like unsubscribing observables and clearing timers.

What is the difference between constructor and ngOnInit?

1. Constructor (Actual Meaning)

👉 Constructor is **NOT Angular-specific**

It is a **TypeScript/JavaScript class feature**

💎 When it runs?

- When Angular **creates the component instance (object)**
- This happens **before Angular does anything else**

Main Purpose:

- **Dependency Injection (MOST IMPORTANT)**
- Basic variable initialization (light work only)

Example:

```
constructor(private userService: UserService) {  
  console.log('Constructor called');  
}
```

✅ Angular creates component → constructor runs immediately

2. ngOnInit (Actual Meaning)

👉 ngOnInit is an **Angular Lifecycle Hook**

◆ When it runs?

- After Angular:
 - Creates the component
 - Sets all `@Input()` values
 - Initializes data bindings

Main Purpose:

- Component initialization
- API calls
- Business logic

Example:

```
ngOnInit(): void {  
  console.log('ngOnInit called');  
  this.loadUsers();  
}
```

Core Differences (Refined Table)

Feature	Constructor	ngOnInit
Type	TypeScript / JS	Angular Hook
Who calls it?	JS Engine	Angular

When runs?	Object creation	After initialization
Dependency Injection	✅ Yes (primary use)	❌ No
@Input available?	❌ No	✅ Yes
API calls	❌ Not safe	✅ Recommended
Business logic	❌ Avoid	✅ Yes

❌ Why NOT constructor for API calls?

At constructor time:

- Angular **has not set** `@Input()` values
- Change detection not started
- Component not ready

👉 You may get:

undefined data ❌

unexpected behavior ❌

Why `ngOnInit` is PERFECT?

At `ngOnInit`:

- All `@Input()` values are set
- Component is initialized
- Change detection is ready

👉 So:

✓ Safe

✓ Predictable

✓ Recommended

Real Example (Very Important)

Parent:

```
<app-child [message]='Hello'></app-child>
```

Child:

```
@Input() message!: string;

constructor() {
  console.log(this.message); // ✗ undefined
}

ngOnInit() {
  console.log(this.message); // ✓ Hello
}
```

👉 This is the **real interview-winning example**

Perfect Interview Answer (Final Version)

If interviewer asks:

👉 **"Difference between constructor and ngOnInit?"**

You can confidently say:

Constructor is a TypeScript feature used for dependency injection and runs when the component instance is created.

ngOnInit is an Angular lifecycle hook that runs after Angular initializes the component and sets input properties.

We should use constructor only for injecting dependencies, and ngOnInit for initialization logic like API calls and business logic.

Final Memory Trick

👉 **Constructor = Injection**

👉 **ngOnInit = Initialization**

What is ViewChild and ContentChild?

👉 Angular components have **2 types of DOM access**:

Type	Meaning
View	Component's own HTML

Content	HTML passed from parent (via <ng-content>)
----------------	--

1. @ViewChild

What it means

@ViewChild is used to access **elements/components inside the component's own HTML (view)**.

Simple definition:

 *"Access something inside my component template"*

Example

HTML

```
<h1 #title>Hello Angular</h1>
<button (click)="changeText()">Click</button>
```

TS

```
import { Component, ViewChild, ElementRef } from
 '@angular/core';

@Component({
```

```
    selector: 'app-demo',
    templateUrl: './demo.component.html'
  })
  export class DemoComponent {

    @ViewChild('title') title!: ElementRef;

    changeText() {
      this.title.nativeElement.innerText = 'Changed!';
    }
  }
```

What happens here?

- #title → template reference variable
- @ViewChild('title') → grabs that element
- You can now manipulate DOM

When is it available?

👉 After:

ngAfterViewInit()

✓ Because Angular must first render the view

Real-world analogy

👉 Your component = your house

👉 ViewChild = accessing things **inside your house**

2. @ContentChild

What it means

@ContentChild is used to access **content projected from parent component** using <ng-content>.

Simple definition:

 "Access content that comes from parent into my component"

Example

Parent Component

```
<app-child>
  <p #msg>Hello from Parent</p>
</app-child>
```

Child Component HTML

```
<ng-content></ng-content>
```

Child Component TS

```
import { Component, ContentChild, ElementRef,
AfterContentInit } from '@angular/core';

@Component({
  selector: 'app-child',
  templateUrl: './child.component.html'
})
export class ChildComponent implements
```

```
AfterContentInit {  
    @ContentChild('msg') message!: ElementRef;  
    ngAfterContentInit() {  
        console.log(this.message.nativeElement.innerText);  
    }  
}
```

What happens here?

- Parent sends content → <p #msg>
- Child receives via <ng-content>
- @ContentChild accesses that projected content

When is it available?

👉 After:

ngAfterContentInit()

✅ Because Angular must first project the content

Real-world analogy

👉 Parent sends furniture into your house

👉 ContentChild = accessing **that external furniture**

Key Difference (VERY IMPORTANT)

Feature	ViewChild	ContentChild
Accesses	Own template	Projected content
Comes from	Same component	Parent component
Lifecycle	ngAfterViewInit	ngAfterContentInit
Uses	DOM / child components	ng-content

Visual Understanding

Parent Component

↓ sends content

```
<app-child>
```

```
  <p>Projected</p>
```

```
</app-child>
```

Child Component

Own HTML (View)

↓ ViewChild

Projected HTML

↓ ContentChild

Important Interview Points

👉 1. You **cannot access ViewChild** in **ngOnInit**

👉 2. You **cannot access ContentChild** in **ngOnInit**

Because:

- View not ready ❌
- Content not projected ❌

One-Line Difference (Interview Ready)

👉 **ViewChild** → Access elements inside component template

👉 **ContentChild** → Access elements passed from parent via ng-content

What is component communication in Angular?

Component communication in Angular is the way **different components share data and interact with each other**. Since Angular apps are built using a **component-based architecture**, components often need to pass data, trigger actions, or stay in sync.

Let's break it down in a clear, structured way 👉

Why Component Communication is Needed

In real applications:

- A **parent component** sends data to a **child**
- A **child component** notifies the **parent**
- Two unrelated components need to share data

Without communication, components would be isolated and useless in real apps.

Types of Component Communication in Angular

Angular supports multiple communication patterns depending on the relationship between components.

1 Parent → Child Communication

Using @Input()

The parent sends data to the child using property binding.

Example:

👉 Child Component

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `<p>{{ message }}</p>`
})
```

```
export class ChildComponent {  
  @Input() message!: string;  
}
```

👉 Parent Component

```
<app-child [message]="parentMessage"></app-child>  
export class ParentComponent {  
  parentMessage = "Hello from Parent!";  
}
```

✓ Key Idea:

- @Input() allows **data flow from parent to child**

2 Child → Parent Communication

Using @Output() and EventEmitter

Child sends events/data back to parent.

Example:

👉 Child Component

```
import { Component, Output, EventEmitter } from  
'@angular/core';  
  
@Component({  
  selector: 'app-child',
```



```
    template: `<button  
(click)="sendData()">Send</button>`  
  })  
  export class ChildComponent {  
    @Output() notify = new EventEmitter<string>();  
  
    sendData() {  
      this.notify.emit("Hello Parent!");  
    }  
  }  
}
```

Parent Component

```
<app-child (notify)="receiveData($event)"></app-  
child>  
export class ParentComponent {  
  receiveData(message: string) {  
    console.log(message);  
  }  
}
```

Key Idea:

- @Output() enables **child to parent communication via events**

3 Parent ↔ Child (Direct Access)

Using @ViewChild()

Parent directly accesses child component methods or properties.

Example:

```
@ViewChild(ChildComponent) child!: ChildComponent;

ngAfterViewInit() {
  this.child.someMethod();
}
```

✓ Key Idea:

- Used when parent needs **direct control over child**
- Not recommended for heavy usage (tight coupling)

4 Content Projection Communication

Using @ContentChild()

Used when components project content using <ng-content>.

✓ Key Idea:

- Access projected content inside a component

5 Sibling / Unrelated Components

When components are not directly related:

Using a Shared Service (Most Important 🏆)

This is the **most commonly used approach in real applications**.

Example:

👉 Service

```
import { Injectable } from '@angular/core';
import { Subject } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class DataService {
  private messageSource = new Subject<string>();
  message$ = this.messageSource.asObservable();

  sendMessage(msg: string) {
    this.messageSource.next(msg);
  }
}
```

Component A

```
constructor(private dataService: DataService) {}

send() {
  this.dataService.sendMessage("Hello from A");
}
```

Component B

```
constructor(private dataService: DataService) {}

ngOnInit() {
  this.dataService.message$.subscribe(msg => {
    console.log(msg);
  });
}
```

✓ Key Idea:

- Uses **RxJS Observables**
- Ideal for **decoupled communication**

What is Data Binding in Angular?

Data binding is the mechanism that keeps your **UI (view)** and **component (model)** in sync automatically.

👉 In simple words:

- If data changes in the component → UI updates
- If user interacts with UI → component updates

This avoids manually manipulating the DOM.

Why Data Binding is Important

Without data binding:

- You would manually update UI using JavaScript
- Code becomes complex and hard to maintain

With Angular:

- UI automatically reflects changes
- Clean separation of logic and view

What is Interpolation?

Interpolation uses **double curly braces** `{{ }}` to bind and display values.

👉 In simple words:

It lets you **embed component data directly into HTML**.

Basic Example

👉 Component

```
export class AppComponent {  
  name = "Lakshmi";  
}
```

👉 Template

```
<p>Hello {{ name }}</p>
```

✓ Output:

```
Hello Lakshmi
```

How It Works

- Angular reads `{{ name }}`
- Looks for name in the component
- Inserts the value into the DOM
- Updates automatically when the value changes

What You Can Use Inside Interpolation

Interpolation is not limited to variables—you can use:

✓ 1. Variables

```
{{ name }}
```

✓ 2. Expressions

```
{{ 10 + 20 }}    <!-- Output: 30 -->
```

✓ 3. Method Calls

```
{{ getMessage() }}  
getMessage() {  
  return "Hello!";  
}
```

```
}
```

✓ 4. String Operations

```
{{ name.toUpperCase() }}
```

Simple Analogy

Think of interpolation like a **placeholder**:

"Hello {{ name }}" → fill in the blank dynamically

what is property binding:

Simple Definition

Property binding allows you to **bind a value from your component to an HTML element's property** using square brackets [].

Syntax

```
[elementProperty]="componentProperty"
```

Example

Component (TypeScript)

```
export class AppComponent {  
  imageUrl = "https://example.com/image.jpg";  
  isEnabled = true;  
}
```

```
}
```

Template (HTML)

```
<img [src]="imageUrl">  
<button [disabled]="isDisabled">Click Me</button>
```

How It Works (Step-by-Step)

1. Angular reads the value from the component (`imageUrl`, `isDisabled`)
2. It updates the **DOM property** (`src`, `disabled`)
3. Whenever the component value changes → UI automatically updates

Important Points

-  **One-way binding** (Component → View)
-  Uses **square brackets** []
-  Works with **DOM properties**, not HTML attributes
-  Dynamic updates reflect instantly in UI

Property vs Attribute (Very Important Concept)

Property Binding	Attribute
Works with DOM properties	Works with HTML attributes

Dynamic (updates automatically)	Static (initial value only)
Example: <code>[disabled]="true"</code>	Example: <code>disabled="true"</code>

👉 Angular always prefers **properties**, because they reflect real-time state.

Real-Life Analogy

Think of property binding like a **remote control**:

- Your component = Remote
- UI element = TV
- When you press a button (change value), the TV updates instantly

When to Use Property Binding

Use it when you want to:

- Set element values dynamically (src, href, value)
- Enable/disable elements
- Bind classes, styles, or other properties

Property Binding vs Interpolation in Angular

Both are used to **bind data from the component to the view**, but they work differently and are used in different situations.

1. Property Binding

✓ Definition

Property binding is used to **set DOM element properties** using square brackets [].

✓ Syntax

```
[elementProperty]="value"
```

✓ Example

```
<img [src]="imageUrl">  
<button [disabled]="isDisabled">Click</button>
```

✓ Key Point

- Binds to **DOM properties**
- Used when Angular needs to interact with element behavior

2. Interpolation

✓ Definition

Interpolation is used to **display data inside HTML** using double curly braces {{ }}.

✓ Syntax

```
{{ expression }}
```

✓ Example

```
<h1>{{ title }}</h1>  
<p>{{ userName }}</p>
```

✓ Key Point

- Used for **displaying text content**
- Converts value to string automatically

Core Differences (Very Important)

Feature	Property Binding	Interpolation
Syntax	[property]="value"	{{ value }}
Used for	DOM properties	Displaying text
Output type	Actual value (boolean, object, etc.)	Always string
Example	[disabled]="true"	{{ true }}

Use case	Control behavior	Show data
----------	------------------	-----------

Real Example to Compare

Component

```
export class AppComponent {  
  title = "Angular App";  
  isDisabled = true;  
}
```

Template

```
<h1>{{ title }}</h1> <!-- Interpolation -->  
  
<button [disabled]="isDisabled">Click</button> <!--  
Property Binding -->
```

Important Interview Point

👉 Interpolation can also be used inside property binding (Angular converts it internally)

```

```

BUT internally Angular treats it like:

```
<img [src]="imageUrl">
```

✓ However, **best practice**:

- Use **property binding** for attributes like src, href, disabled
- Use **interpolation** for text content

Simple Analogy

- **Interpolation** → Showing data (like printing text on screen)
- **Property Binding** → Controlling behavior (like enabling/disabling a button)

When to Use What

Scenario	Use
Display text	Interpolation
Set image source	Property binding
Disable button	Property binding
Show username	Interpolation

What is event binding?

What is Event Binding in Angular?

Event binding is used to **listen to user actions (events) in the UI** and execute a method in your component (TypeScript).

👉 In simple terms:

Event binding = View → Component communication

Syntax

```
(eventName)="methodName()"
```

Example

Component (TypeScript)

```
export class AppComponent {  
  onClick() {  
    console.log("Button clicked!");  
  }  
}
```

Template (HTML)

```
<button (click)="onClick()">Click Me</button>
```

How It Works (Step-by-Step)

1. User performs an action (click, input, hover, etc.)
2. Angular listens to that event
3. It calls the specified method in the component
4. Your logic gets executed

Common Events

Event	Description
(click)	Button click
(input)	Typing in input field
(change)	Value change
(mouseover)	Mouse hover
(keydown)	Key press

Passing Data from Event

You can pass values using `$event`.

```
<input (input)="onInput($event)">
onInput(event: any) {
  console.log(event.target.value);
}
```

Example with Value Binding

```
<input (input)="updateName($event)">
<p>{{ name }}</p>
name = '';

updateName(event: any) {
  this.name = event.target.value;
}
```

👉 Here:

- User types → event triggers → component updates → UI updates

Important Points

-  Uses **round brackets ()**
-  It is **one-way binding (View → Component)**
-  Used for handling **user interactions**
-  Works with all DOM events

Property Binding vs Event Binding

Feature	Property Binding	Event Binding
Direction	Component → View	View → Component
Syntax	[property]	(event)

Example	[disabled]="true"	(click)="onClick()"
---------	-------------------	---------------------

Real-Life Analogy

- **Property Binding** → You send instructions to UI
- **Event Binding** → UI sends feedback to you

When to Use Event Binding

Use it when you want to:

- Handle button clicks
- Capture user input
- Respond to keyboard/mouse actions
- Trigger business logic from UI

What is Two-Way Data Binding?

Two-way data binding is a technique where **data flows in both directions**:

👉 **Component ⇌ View (UI)**

- When data in the **component changes** → **UI updates**
- When user changes something in the **UI** → **component updates**

Syntax

```
[(ngModel)]="property"
```

👉 This is called **banana in a box syntax** 🍌

(because of [(]))

Example

Component (TypeScript)

```
export class AppComponent {  
  name = '';  
}
```

Template (HTML)

```
<input [(ngModel)]="name">  
<p>Your name is: {{ name }}</p>
```

How It Works (Step-by-Step)

1. Initial value of name is shown in input
2. User types in the input field
3. Angular updates the name in the component
4. The updated value is reflected instantly in UI

👉 Both sides stay in sync automatically

Behind the Scenes (Very Important)

Two-way binding is actually a **combination of:**

- **Property binding** → [value]="name"
- **Event binding** → (input)="name = `$event.target.value`"

👉 Angular combines both into:

`[(ngModel)]="name"`

Prerequisite ⚠️

To use `ngModel`, you must import:

```
import { FormsModule } from '@angular/forms';
```

Important Points

-  Uses `[( )]` syntax
-  Works mainly with form elements (input, select, etc.)
-  Keeps UI and component always synchronized
-  Simplifies form handling

Real-Life Analogy

Think of it like a **live chat**:

- You type → message updates instantly
- System updates → you see it instantly

👉 Both sides communicate continuously

Comparison with Other Bindings

Binding Type	Direction	Syntax
--------------	-----------	--------

Interpolation	Component → View	{{ }}
Property Binding	Component → View	[]
Event Binding	View → Component	()
Two-way Binding	Both directions	[()]

When to Use Two-Way Binding

Use it when:

- Handling **forms**
- Capturing **user input**
- Keeping UI and data always in sync

Data Binding vs Directives in Angular

Data Binding and Directives are both core concepts in Angular, but they serve different purposes.

Data Binding is used to establish communication between the component (TypeScript) and the view (HTML). It defines how data flows between them. Angular provides four types of data binding: interpolation, property binding, event binding, and two-way binding.

On the other hand, **Directives** are used to manipulate the DOM by adding, removing, or modifying elements and their behavior. They instruct Angular on how to render or change the UI.

The key difference is:

- Data binding deals with **data flow**
- Directives deal with **DOM manipulation and behavior**

For example:

- Using `{{name}}` displays data → Data Binding
- Using `*ngIf` conditionally shows or hides elements → Directive

In simple terms,

Data Binding connects data to the UI, while Directives control how the UI behaves.

Services & Dependency Injection in Angular (Complete Guide)

Angular applications are built using **components**, but components should not contain all logic.

To keep code **clean, reusable, and maintainable**, Angular uses **services + dependency injection (DI)**.

What is a Service in Angular?

Definition

A **service** is a class used to **store reusable business logic, data access logic, or shared functionality**.

👉 Instead of writing logic inside components, we move it into services.

Example

```
@Injectable({
  providedIn: 'root'
})
export class UserService {
  getUsers() {
    return ['Alice', 'Bob', 'Charlie'];
  }
}
```

Why Services?

-  Reusability across components
-  Separation of concerns
-  Cleaner components
-  Easier testing

Real-Life Analogy

👉 Component = UI (like a waiter)

👉 Service = Backend logic (like a kitchen)

The waiter doesn't cook — it asks the kitchen (service).

What is Dependency Injection (DI)?

Definition

Dependency Injection is a design pattern where a class receives its dependencies **from outside**, instead of creating them itself.

Without DI (Bad Practice ✖)

```
const service = new UserService(); // tightly coupled
```

With DI (Angular Way ✔)

```
constructor(private userService: UserService) {}
```

👉 Angular automatically provides the instance.

Benefits

- ✔ Loose coupling
- ✔ Better testing
- ✔ Reusability
- ✔ Maintainability

Simple Understanding

👉 Instead of **creating objects**, Angular **injects them**

What is an Injector?

An **Injector** in Angular is the **core mechanism that creates and provides dependencies (services) to components and other classes.**

Let's break it down clearly so you can understand it deeply 🙌

What is an Injector?

An **Injector** is an object responsible for:

- 🙌 Creating instances of services
- 🙌 Storing those instances
- 🙌 Providing them whenever requested

In simple words:

Injector = Service factory + Service container

Why do we need an Injector?

In Angular, we don't manually create services like this:

```
const service = new MyService(); ❌
```

Instead, Angular does it for us using **Dependency Injection (DI)**.

- 🙌 The **Injector** handles:

- When to create a service
- How many instances to create
- Where to share the instance

How Injector Works (Step-by-Step)

Let's understand the flow:

1. You define a service

```
@Injectable({
  providedIn: 'root'
})
export class DataService {
  getData() {
    return "Hello from service";
  }
}
```

2. You request it in a component

```
@Component({...})
export class AppComponent {
  constructor(private dataService: DataService) {}
}
```

3. Injector comes into action

👉 Angular checks:

- “Is there already an instance of DataService?”

👉 If YES → reuse it

👉 If NO → create new instance

👉 Then it injects into the component constructor

Types of Injectors in Angular

Angular uses **Hierarchical Dependency Injection**, meaning multiple injectors exist.

1. Root Injector

- Created at application start
- Shared across entire app
- Used when:

```
providedIn: 'root'
```

👉 One instance for the whole app (Singleton)

2. Component-Level Injector

Defined in component:

```
@Component({  
  providers: [DataService]  
})
```

👉 Creates a **new instance per component**

3. Module Injector (less used in modern Angular)

Defined in NgModule providers.

Hierarchical Behavior (Very Important)

Injectors form a **tree structure**:

```
Root Injector
  ↓
Module Injector
  ↓
Component Injector
  ↓
Child Component Injector
```

👉 Angular searches **bottom → top**:

1. Check current component injector
2. If not found → parent
3. Continue → until root

Injector is a core part of Angular's Dependency Injection system that is responsible for creating, managing, and providing service instances to components and other classes. It maintains a hierarchy and ensures services are reused or created as needed.

What is Hierarchical Dependency Injection?

Hierarchical Dependency Injection means:

Angular maintains a **tree of injectors**, and when a dependency (service) is requested, Angular searches for it **from the current level upward** in the hierarchy.

Simple Definition (Interview Ready)

Hierarchical Dependency Injection is a system in Angular where multiple injectors are organized in a tree structure, and Angular resolves dependencies by searching from the current injector up to parent injectors until the service is found.

Why Angular Uses Hierarchy?

This allows:

- ✓ Service sharing across components
- ✓ Different instances in different parts of app
- ✓ Better memory usage
- ✓ Scoped services (local or global)

Injector Hierarchy Structure

Think of Angular injectors like a **tree**:

```
Root Injector (App Level)
  ↓
Module Injector
  ↓
Component Injector (Parent)
  ↓
Component Injector (Child)
```

How Resolution Works (Very Important)

When a component asks for a service:

Step-by-step:

1. Check **current component injector**
2. If not found → check **parent injector**
3. Continue upward
4. Finally → check **root injector**

👉 If found → use it

👉 If not → error

Example to Understand Clearly

Service

```
@Injectable({  
  providedIn: 'root'  
})  
export class LoggerService {}
```

Parent Component

```
@Component({  
  selector: 'app-parent',  
  providers: [LoggerService] // New instance here  
})  
export class ParentComponent {}
```

Child Component

```
@Component({  
  selector: 'app-child'  
})  
export class ChildComponent {  
  constructor(private logger: LoggerService) {}  
}
```

What Happens Internally?

👉 ChildComponent asks for LoggerService

Angular checks:

1. Child Injector → ❌ Not found
2. Parent Injector → ✅ Found

👉 So **child uses parent's instance**, NOT root

Important Behavior (Most Asked in Interviews)

Case 1: providedIn: 'root'

👉 Single instance across app (Singleton)

Case 2: providers in Component

👉 New instance per component

Case 3: Multiple Levels

Parent → providers: [Service]
Child → providers: [Service]

👉 Child gets **its own instance**

👉 Parent has **different instance**

Real-Life Analogy

Think of it like a **family hierarchy** 👨👩👧

- Child asks for money 💰
- First asks parents
- If not → grandparents
- If found → uses it

Key Points to Remember

- ✓ Angular uses **injector tree**
- ✓ Resolution is **bottom → top**
- ✓ Closest provider wins
- ✓ Enables **service scoping**
- ✓ Supports **multiple instances**

Short Interview Answer

Hierarchical Dependency Injection in Angular is a system where injectors are arranged in a tree structure, and Angular resolves dependencies by searching from the current injector upward to parent injectors. The nearest available provider is used, allowing services to be scoped at different levels like root, module, or component.

What is the difference between `providedIn: 'root'` and `providers` array?

1. `providedIn: 'root'`

Definition

```
@Injectable({  
  providedIn: 'root'  
})
```



```
export class UserService {}
```

What it means

👉 Angular registers this service in the **Root Injector**

👉 Only **ONE instance** is created for the entire application

Behavior

- Shared across all components
- Lazy-loaded automatically when needed
- Tree-shakable (removed if unused)

Example

```
@Component({...})  
export class AComponent {  
  constructor(private user: UserService) {}  
}  
  
@Component({...})  
export class BComponent {  
  constructor(private user: UserService) {}  
}
```

👉 Both components get the **same instance**

When to Use

✓ API services

✓ Auth services

✓ Global state services

2. providers Array

✓ Definition (Component level)

```
@Component({  
  providers: [UserService]  
})  
export class MyComponent {}
```

What it means

👉 Angular creates a **new injector at this level**

👉 A **new instance** of service is created

Behavior

- Each component gets its own instance
- Not shared globally
- Child components can inherit it

Example

```
@Component({  
  selector: 'parent',  
  providers: [UserService]  
})  
export class ParentComponent {}
```

```
@Component({
  selector: 'child'
})
export class ChildComponent {
  constructor(private user: UserService) {}
}
```

👉 Child uses **Parent's instance**

1. Scope

providedIn: 'root' → App-wide

providers → Limited to that component/module

2. Instance Creation

Root → Singleton

Providers → Multiple instances possible

How do services share data between components?

Core Idea

Services share data between components by acting as a **central data store**, and Angular's Dependency Injection ensures that the **same service instance is used** across components (when provided in root).

How It Works (Concept)

1. A **service** holds the data
2. Multiple components **inject the same service**
3. Since the service is **singleton**, all components access the same data

Basic Example (Step-by-Step)

1. Create Service

```
@Injectable({
  providedIn: 'root'
})
export class DataService {
  message: string = "Initial Message";
}
```

2. Sender Component (updates data)

```
@Component({...})
export class SenderComponent {
  constructor(private dataService: DataService) {}

  sendMessage() {
    this.dataService.message = "Hello from Sender!";
  }
}
```

3. Receiver Component (reads data)

```
@Component({...})
export class ReceiverComponent {
  constructor(public dataService: DataService) {}
}
<p>{{ dataService.message }}</p>
```

What Happens Internally?

- 👉 Both components use the **same DataService instance**
- 👉 When Sender updates → Receiver automatically sees updated value

Problem with Simple Approach

The above works, but:

- ❌ No real-time updates
- ❌ Not reactive
- ❌ Change detection issues in complex apps

Best Practice: Use RxJS (Reactive Approach)

- 👉 This is what interviewers expect 👉

Using BehaviorSubject

1. Service

```
import { BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {

  private messageSource = new
BehaviorSubject<string>("Initial Message");

  currentMessage$ =
this.messageSource.asObservable();

  updateMessage(msg: string) {
    this.messageSource.next(msg);
  }
}
```

2. Sender Component

```
export class SenderComponent {
  constructor(private dataService: DataService) {}

  sendMessage() {
    this.dataService.updateMessage("Hello from
Sender!");
  }
}
```

```
}
```

3. Receiver Component

```
export class ReceiverComponent {  
  message!: string;  
  
  constructor(private dataService: DataService) {}  
  
  ngOnInit() {  
    this.dataService.currentMessage$.subscribe(msg =>  
    {  
      this.message = msg;  
    });  
  }  
}
```

Why BehaviorSubject?

- ✓ Holds latest value
- ✓ Emits to new subscribers immediately
- ✓ Enables real-time updates

Data Flow Visualization

Sender Component



DataService (BehaviorSubject)



Receiver Component

Important Rule

👉 For sharing data:

- Service must be **singleton**

`providedIn: 'root'`

❗ If you use providers in component → data will NOT be shared

Interview-Ready Answer

Services share data between components by acting as a shared state container. When a service is provided in the root injector, Angular creates a single instance that can be injected into multiple components. Using RxJS subjects like BehaviorSubject allows real-time data sharing and updates across components.

What is Angular routing?

Angular routing is the mechanism that allows you to navigate between different views (components) in a **Single Page Application (SPA)** without reloading the entire page.

Instead of loading a new HTML page from the server (like traditional websites), Angular dynamically swaps components based on the URL.

1. What is Angular Routing (Simple Understanding)

Think of routing like this:

👉 URL changes → Angular decides which component to show

Example:

```
/home    → HomeComponent  
/about   → AboutComponent  
/profile → ProfileComponent
```

So Angular routing:

- Maps **URLs** → **Components**
- Enables **navigation without page reload**
- Makes your app behave like a real website

2. Why Do We Need Routing?

Without routing:

- Entire page reloads every time
- Poor user experience
- Slower performance

With Angular routing:

- Fast navigation ⚡

- No full page reload
- Maintains application state

3. Core Building Blocks of Angular Routing

Let's understand the important pieces.

3.1 Routes Configuration

You define routes in an array:

```
import { Routes } from '@angular/router';
import { HomeComponent } from
'./home/home.component';
import { AboutComponent } from
'./about/about.component';

export const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent }
];
```

👉 This tells Angular:

- If URL is /home → load HomeComponent
- If URL is /about → load AboutComponent

3.2 RouterModule

To activate routing:

```
import { RouterModule } from '@angular/router';

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

👉 `forRoot()`:

- Registers routes
- Sets up router service

3.3 router-outlet (Very Important)

This is where Angular loads components dynamically.

```
<router-outlet></router-outlet>
```

👉 Think of it as:

"Placeholder where routed component will be displayed"

3.4 routerLink (Navigation)

Used in HTML to navigate:

```
<a routerLink="/home">Home</a>
<a routerLink="/about">About</a>
```

👉 No page reload — Angular handles navigation internally.

4. How Routing Works (Step-by-Step Flow)

1. User clicks link (routerLink)
2. URL changes (e.g., /home)
3. Angular Router matches route
4. Corresponding component is loaded
5. Component is rendered inside <router-outlet>

5. Example (Complete Flow)

Step 1: Define Routes

```
const routes: Routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'about', component: AboutComponent }  
];
```

Step 2: Add Router Outlet

```
<h1>My App</h1>  
  
<nav>  
  <a routerLink="/home">Home</a>  
  <a routerLink="/about">About</a>  
</nav>  
  
<router-outlet></router-outlet>
```

Step 3: Result

- Click **Home** → HomeComponent loads
- Click **About** → AboutComponent loads

What is the purpose of RouterModule?

The **RouterModule** is the core module in Angular that **enables routing (navigation)** in your application.

Instead of just defining routes, Angular needs something to **activate and manage them** — that's exactly what RouterModule does.

1. Simple Definition

👉 RouterModule is responsible for configuring and providing routing functionality in an Angular application.

It:

- Registers routes
- Enables navigation
- Provides routing directives (routerLink, router-outlet)
- Manages URL changes

2. Why Do We Need RouterModule?

If you only write routes like this:

```
const routes = [  
  { path: 'home', component: HomeComponent }  
];
```

❌ Nothing will work yet!

👉 Because Angular doesn't know:

- How to use these routes
- How to listen to URL changes
- Where to render components

✅ That's why we use:

`RouterModule.forRoot(routes)`

3. What RouterModule Actually Does

When you import RouterModule, Angular:

✅ 1. Registers Routes

- Stores route configuration internally

✅ 2. Listens to URL Changes

- Detects when URL changes (/home, /about)

✅ 3. Loads Components Dynamically

- Matches URL → loads corresponding component

✅ 4. Provides Directives

You can use:

- `routerLink` → navigation
- `router-outlet` → display component

✅ 5. Provides Router Services

- Router (navigate programmatically)

- ActivatedRoute (read route data)

5. Example (Complete Understanding)

Step 1: Define Routes

```
const routes = [  
  { path: 'home', component: HomeComponent }  
];
```

Step 2: Import RouterModule

```
import { RouterModule } from '@angular/router';  
  
@NgModule({  
  imports: [RouterModule.forRoot(routes)],  
  exports: [RouterModule]  
})  
export class AppRoutingModule {}
```

Step 3: Use in Template

```
<a routerLink="/home">Home</a>  
  
<router-outlet></router-outlet>
```

👉 Now everything works because RouterModule:

- Knows routes
- Handles navigation
- Renders component

6. Real-Life Analogy

Think of routing like **Google Maps**:

- Routes = Locations
- RouterModule = Navigation system
- URL = Address
- Component = Destination

👉 Without RouterModule → You have locations but no navigation system

7. Interview-Ready Answer

👉 RouterModule is an Angular module that enables routing by registering route configurations, handling navigation, and providing directives like routerLink and router-outlet. It connects URLs to components and manages dynamic view loading in a single-page application.

What is router-outlet?

What is router-outlet in Angular?

👉 router-outlet is a directive that acts as a placeholder where Angular dynamically loads and displays routed components.

1. Simple Understanding

Think of it like this:

👉 URL changes → Angular finds component → loads it inside router-outlet

```
<router-outlet></router-outlet>
```

✓ This is the **area of your UI that changes when navigation happens**

2. Why Do We Need router-outlet?

Without router-outlet:

✗ Angular knows which component to load

✗ But doesn't know **where to display it**

👉 So nothing appears on screen

With router-outlet:

✓ Angular gets a **target location to render components**

3. How It Works (Step-by-Step Flow)

Let's say you have routes:

```
const routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'about', component: AboutComponent }  
];
```

And HTML:

```
<a routerLink="/home">Home</a>
<a routerLink="/about">About</a>

<router-outlet></router-outlet>
```

Flow:

1. User clicks **Home**
2. URL becomes /home
3. Angular Router matches route
4. Loads HomeComponent
5. Displays it inside <router-outlet>

4. Example (Clear Visualization)

```
<h1>My App</h1>

<nav>
  <a routerLink="/home">Home</a>
  <a routerLink="/about">About</a>
</nav>

<!-- Dynamic content loads here -->
<router-outlet></router-outlet>
```

👉 Output:

- Click **Home** → HomeComponent appears
- Click **About** → AboutComponent appears

5. Multiple router-outlets (Advanced)

You can even have multiple outlets:

```
<router-outlet></router-outlet>  
<router-outlet name="sidebar"></router-outlet>
```

Routes:

```
{  
  path: 'home',  
  component: HomeComponent  
},  
{  
  path: 'menu',  
  component: MenuComponent,  
  outlet: 'sidebar'  
}
```

👉 This allows:

- Main content + sidebar rendering

6. Important Points

- ✓ It is a **directive**, not a component
- ✓ It works with RouterModule
- ✓ It loads components dynamically
- ✓ Required for routing to display anything

8. Interview-Ready Answer

👉 **router-outlet** is a directive in Angular that serves as a placeholder where routed components are dynamically rendered based on the current URL. It acts as the view container for navigation.

What is routerLink in Angular?

👉 **routerLink** is a directive used in Angular templates to navigate between routes (components) without reloading the page.

It binds a URL path to an HTML element (like `<a>` or `<button>`) and tells Angular:

“When this is clicked, navigate to this route”

1. Simple Understanding

```
<a routerLink="/home">Home</a>
```

👉 When user clicks:

- URL becomes /home
- Angular loads HomeComponent
- Page does NOT reload

2. Why Do We Use routerLink?

Without routerLink:

```
<a href="/home">Home</a>
```

✗ Full page reload happens

✗ Angular app restarts

With routerLink:

```
<a routerLink="/home">Home</a>
```

✓ No reload

✓ Smooth navigation

✓ SPA behavior

3. How routerLink Works Internally

1. User clicks element
2. Angular intercepts click
3. Prevents default browser reload
4. Updates URL
5. Router loads corresponding component
6. Component is shown inside router-outlet

4. Basic Examples

Static Route

```
<a routerLink="/about">About</a>
```

Dynamic Route (with parameters)

```
<a [routerLink]="['/user', 101]">User 101</a>
```

👉 Navigates to:

```
/user/101
```

Using with Button

```
<button routerLink="/contact">Go to Contact</button>
```

What are Route Parameters?

👉 Route parameters are values passed as part of the URL path to identify a specific resource.

They are **required** and defined inside the route.

Example

Route Configuration

```
{ path: 'user/:id', component: UserComponent }
```

👉 `:id` is a **route parameter**

URL

```
/user/101
```

👉 Here:

- `id = 101`

Access in Component

```
import { ActivatedRoute } from '@angular/router';

constructor(private route: ActivatedRoute) {}

ngOnInit() {
  const id = this.route.snapshot.paramMap.get('id');
  console.log(id); // 101
}
```

Key Points

- ✓ Part of URL path
- ✓ Used to identify specific data (user, product, etc.)
- ✓ Mandatory for that route

What are Query Parameters?

👉 Query parameters are optional key

-value pairs appended to the URL after ? used to pass additional data.

Example

URL

```
/products?category=books&sort=price
```

👉 Here:

- category = books
- sort = price

Route Configuration

```
{ path: 'products', component: ProductsComponent }
```

👉 No need to define query params in route

Access in Component

```
ngOnInit() {  
  this.route.queryParams.subscribe(params => {  
    console.log(params['category']); // books  
  });  
}
```

Key Points

- ✓ Not part of route path
- ✓ Optional
- ✓ Used for filters, sorting, pagination

3. Route Params vs Query Params (Clear Difference)

Feature	Route Parameters	Query Parameters
URL format	/user/101	/products?category=books

Defined in route	 Yes	 No
Mandatory	 Usually	 Optional
Purpose	Identify resource	Pass extra info
Example use	User ID, Product ID	Filters, search, sort

Real-Time Use Case

User Profile Page

/user/101

👉 Use **route param**

- Because user ID is required

Product Listing Page

/products?category=books&sort=price

👉 Use **query params**

- Because filters are optional

5. Combined Example (Very Important)

/user/101?tab=posts

👉 Here:

- Route param → id = 101
- Query param → tab = posts

6. Interview-Ready Answer

👉 Route parameters are dynamic values embedded in the URL path used to identify specific resources, while query parameters are optional key-value pairs appended to the URL used to pass additional information like filters or sorting.

What are Child Routes in Angular?

👉 Child routes are routes defined inside another (parent) route, allowing you to create nested navigation and layouts.

They help you build **hierarchical UI structures** like dashboards, admin panels, profiles, etc.

1. Simple Understanding

Think like this:

/dashboard	→ Parent (DashboardComponent)
/dashboard/profile	→ Child (ProfileComponent)
/dashboard/settings	→ Child (SettingsComponent)

👉 The parent component stays **constant**, and only the child part changes.

2. Why Do We Need Child Routes?

Without child routes:

- You reload or replace entire components
- Difficult to manage layouts

With child routes:

- Reuse parent layout (header/sidebar)
- Change only inner content
- Better structure & scalability

3. Basic Example

Step 1: Route Configuration

```
const routes = [  
  {  
    path: 'dashboard',  
    component: DashboardComponent,  
    children: [  
      { path: 'profile', component:  
ProfileComponent },  
      { path: 'settings', component:  
SettingsComponent }  
    ]  
  }  
];
```

Step 2: Parent Template

```
<h2>Dashboard</h2>  
  
<nav>  
  <a routerLink="profile">Profile</a>  
  <a routerLink="settings">Settings</a>
```

```
</nav>

<!-- Child components load here -->
<router-outlet></router-outlet>
```

4. How It Works (Flow)

1. User goes to /dashboard

→ DashboardComponent loads

2. User clicks **Profile**

→ URL becomes /dashboard/profile

3. Angular:

a. Keeps DashboardComponent

b. Loads ProfileComponent inside child <router-outlet>

10. Interview-Ready Answer

👉 Child routes are nested routes defined within a parent route that allow rendering of components inside another component using a nested router-outlet. They are used to create hierarchical navigation and reusable layouts in Angular applications.

What are Forms in Angular?

Think of a form like a **container that holds user inputs**.

For example:

- Username
- Password
- Email
- Address

Angular provides a **structured and powerful way** to:

1. Capture user input
2. Track changes
3. Validate data
4. Submit data

Simple Real-Life Example

Imagine a **login form**:

```
Username: [_____]
Password: [_____]
[ Submit ]
```

Angular helps you:

- Know what the user typed
- Check if fields are empty
- Show errors like “Password is required”
- Send data to backend

Types of Forms in Angular

Angular gives **2 ways to create forms**:

Template-Driven Forms

- Defined mostly in HTML
- Uses directives like `ngModel`
- Simple and beginner-friendly

👉 Example:

```
<form #myForm="ngForm">
  <input type="text" name="username" ngModel>
</form>
```

✓ Easy to use

✗ Not scalable for complex apps

2 Reactive Forms (Recommended 🚀)

- Defined in TypeScript
- Uses classes like `FormGroup`, `FormControl`
- More control and scalable

👉 Example:

```
this.loginForm = new FormGroup({
  username: new FormControl(''),
  password: new FormControl('')
});<form [formGroup]="loginForm">
  <input formControlName="username">
</form>
```

✓ Powerful

- ✓ Better validation
- ✓ Used in real-world apps

What is the difference between Template-driven forms and Reactive forms?

Core Idea First (Very Important)

👉 Both Template-driven and Reactive forms do the same job:

- Collect user input
- Validate data
- Submit data

👉 The **real difference is: where the form logic lives**

- **Template-driven** → **HTML controls everything**
- **Reactive** → **TypeScript controls everything**

1. Template-Driven Forms (HTML-driven)

👉 Form structure and logic are mostly written in **HTML**

Example:

```
<form #form="ngForm">  
  <input name="username" ngModel required>  
</form>
```

How it works

- Uses directives like `ngModel`
- Angular automatically creates form model behind the scenes
- You don't explicitly define form structure in TS

Advantages

- Very easy to start
- Less code
- Good for small/simple forms

Disadvantages

- Hard to manage complex forms
- Validation logic scattered in HTML
- Not very scalable
- Difficult to test

2. Reactive Forms (Model-driven)

👉 Form structure and logic are defined in **TypeScript**

Example:

```
this.form = new FormGroup({
  username: new FormControl('', Validators.required)
});<form [formGroup]="form">
  <input formControlName="username">
</form>
```


How it works

- You explicitly create form model in TS
- Full control over validation, state, and structure
- Uses FormGroup, FormControl

Advantages

- Highly scalable
- Easy to test
- Clean separation (logic in TS, UI in HTML)
- Powerful validation

Disadvantages

- More code
- Slightly harder for beginners

Side-by-Side Comparison (Easy to Remember)

Feature	Template-Driven	Reactive Forms
Where logic lives	HTML	TypeScript
Setup	Simple	Structured
Control	Less	Full control
Validation	HTML-based	TS-based

Scalability	Low	High
Testing	Hard	Easy
Best for	Small forms	Large/complex forms

Real-World Understanding

👉 Use **Template-driven** when:

- Simple login form
- Contact form
- Small projects

👉 Use **Reactive Forms** when:

- Dynamic forms
- Complex validation
- Enterprise apps (most real-world apps)

Interview One-Liner

Template-driven forms rely on directives in the template and are suitable for simple forms, while Reactive forms are model-driven, defined in TypeScript, and provide better control, scalability, and testability.

Final Intuition (Don't Forget This)

👉 Template-driven = *Angular does the work for you*

👉 Reactive = *You control everything*

What is FormGroup in Angular?

👉 **FormGroup** is a collection of **FormControls** that represents a **complete form**.

In simple words:

FormGroup = **entire form (container)** that holds multiple input fields

Simple Example

Imagine a **login form**:

```
Username: [_____]
Password: [_____]
```

👉 Angular represents it like this:

```
this.loginForm = new FormGroup({
  username: new FormControl(''),
  password: new FormControl('')
});
```

What FormGroup Does

A FormGroup:

1. Holds multiple fields

Each field is a `FormControl`

2. Tracks the whole form state

`this.loginForm.value`

👉 Output:

```
{
  "username": "",
  "password": ""
}
```

3. Checks validation of entire form

`this.loginForm.valid`

👉 If any field is invalid → whole form becomes invalid

4. Provides methods to manage form

```
this.loginForm.setValue({...})
this.loginForm.patchValue({...})
this.loginForm.reset()
```

Connecting FormGroup to HTML

```
<form [formGroup]="loginForm">
  <input formControlName="username">
  <input formControlName="password">
</form>
```

👉 `[formGroup]` → connects the whole form

👉 `formControlName` → connects individual fields

Internal Structure (Important)

Think of FormGroup like a **tree**

```
FormGroup
├── username (FormControl)
└── password (FormControl)
```

With Validation Example

```
this.loginForm = new FormGroup({
  username: new FormControl('', Validators.required),
  password: new FormControl('', Validators.required)
});
```

👉 Now:

- If username is empty → invalid
- If password is empty → invalid
- So whole form → invalid

Real-Life Analogy

👉 FormGroup = **Google Form / Application Form**

👉 FormControl = **each question field**

Important Points (Interview Focus)

- FormGroup is part of **Reactive Forms**
- It groups multiple controls into a single object
- It tracks value, validity, and state of entire form
- It can contain:

- FormControl
- FormArray
- Nested FormGroup

Interview One-Liner

FormGroup is a collection of FormControls that represents a form, manages its structure, and tracks the overall value and validation state.

What is FormControl?

 **FormControl represents a single input field in a form.**

In simple words:

FormControl = one input box (like username, email, password)

Simple Example

Imagine this input:

```
Username: [_____]
```

 Angular represents it like this:

```
username = new FormControl('');
```

What FormControl Does

A FormControl manages everything about that one field:

1 Holds the value

```
this.username.value
```

👉 Gets current value typed by user

2 Tracks validation

```
this.username.valid  
this.username.invalid
```

3 Tracks user interaction

```
this.username.touched    // user clicked field  
this.username.dirty      // value changed  
this.username.pristine   // not changed
```

4 Supports validation rules

```
username = new FormControl('', Validators.required);
```

👉 Now:

- Empty → invalid
- Filled → valid

Connecting to HTML

```
<input [formControl]="username">
```

👉 This connects input field with FormControl

Example with Validation

```
username = new FormControl('', [  
  Validators.required,  
  Validators.minLength(3)  
]);
```

```
<input [formControl]="username">  
  
<p *ngIf="username.invalid && username.touched">  
  Invalid username  
</p>
```

Internal Understanding

Think like this:

```
FormControl  
├── value  
├── validation rules  
├── status (valid/invalid)  
└── user interaction state
```

Real-Life Analogy

👉 FormControl = **one question in a form**

Example:

- “Enter your name” → one FormControl
- “Enter your email” → another FormControl

Quick Difference Recap

Feature	FormControl
Represents	Single input field
Holds	Value + validation + state
Used in	Reactive Forms
Inside	FormGroup or FormArray

Interview One-Liner

FormControl is a class in Angular that represents an individual form field and tracks its value, validation status, and user interaction.

Final Intuition (Very Important)

👉 FormControl = **atom** (smallest unit)

👉 FormGroup = **molecule** (collection of controls)

What is FormArray?

👉 FormArray is a collection of FormControls (or FormGroups) stored in an array-like structure.

In simple words:

FormArray = dynamic list of form fields

When Do We Use FormArray?

Whenever the number of inputs is **not fixed**

Real-world examples:

- Multiple phone numbers 📞
- Multiple email addresses ✉️
- Skills list in resume 🧠
- Dynamic form fields (Add / Remove buttons)

Simple Example

Imagine this UI:

Phone Numbers :

[1234567890] + Add

[9876543210] ✖ Remove

👉 You don't know how many numbers user will add

👉 So you use **FormArray**

How FormArray Looks Internally

```
phones = new FormArray([
  new FormControl('123'),
  new FormControl('456')
]);
```

Structure

```
FormArray
├─ FormControl (index 0)
```

```
├─ FormControl (index 1)
└─ FormControl (index 2)
```

👉 It uses **index (0,1,2...)** instead of names

Example with FormGroup

```
this.form = new FormGroup({
  phones: new FormArray([
    new FormControl('')
  ])
});
```

HTML Binding

```
<div formArrayName="phones">
  <div *ngFor="let phone of phones.controls; let i =
index">
    <input [formControlName]="i">
  </div>
</div>
```

Important Methods (Very Important)

1 Add new field

```
this.phones.push(new FormControl(''));
```

2 Remove field

```
this.phones.removeAt(index);
```

3 Get all values

```
this.phones.value
```

👉 Output:

```
["123", "456"]
```

Getter (Best Practice)

```
get phones() {  
  return this.form.get('phones') as FormArray;  
}
```

Using FormArray with FormGroup (Advanced)

👉 Each item can also be a FormGroup

Example:

```
this.form = new FormGroup({  
  users: new FormArray([  
    new FormGroup({  
      name: new FormControl(''),  
      age: new FormControl('')  
    })  
  ])  
});
```

Real-Life Analogy

👉 FormArray = **list of repeating sections**

Like:

- Adding multiple addresses in a form
- Adding multiple employees

Difference Summary

Feature	FormControl	FormGroup	FormArray
Represents	Single field	Whole form	Dynamic list
Key type	Name	Name	Index
Use case	One input	Fixed form	Dynamic inputs

Interview One-Liner

FormArray is a dynamic collection of form controls or groups that allows you to manage a variable number of form fields using an index-based structure.

Final Intuition (Very Important)

- 👉 FormControl → single input
- 👉 FormGroup → fixed structure
- 👉 FormArray → dynamic structure

How does form validation work in Angular?

Form validation ensures that user input is correct, complete, and follows rules before submission.

Examples:

- Required fields
- Email format
- Minimum length

Core Idea (Very Important)

Angular validation works by:

Attaching rules (validators) to form controls and continuously checking their state

Where Validation Happens

Validation can happen in **two ways**:

1 Template-Driven Forms (HTML-based)

```
<input name="email" ngModel required email>
```

👉 Validation rules are written in HTML

2 Reactive Forms (TypeScript-based Recommended)

```
email: new FormControl('', [  
  Validators.required,  
  Validators.email  
)
```

👉 Validation rules are written in TS

How Angular Validates (Step-by-Step)

Let's take a simple control:

```
username = new FormControl('', Validators.required);
```

Step 1: User types something

👉 Angular listens to input changes

Step 2: Validator runs

👉 `Validators.required` checks:

- Is value empty?

Step 3: Angular updates status

Each control gets:

```
this.username.valid      // true or false  
this.username.invalid    // opposite  
this.username.errors     // error details
```

Step 4: UI reacts

```
<input [formControl]="username">  
  
<p *ngIf="username.invalid && username.touched">  
  Username is required  
</p>
```

Important States Angular Tracks

Every form control has states:

State	Meaning
valid	passes all validations
invalid	fails validation

touched	user clicked field
untouched	not interacted
dirty	value changed
pristine	not changed

Built-in Validators

Angular provides ready-made validators:

```
Validators.required  
Validators.minLength(3)  
Validators.maxLength(10)  
Validators.email  
Validators.pattern(/regex/)
```

Example (Full Reactive Form)

```
this.form = new FormGroup({  
  username: new FormControl('', [  
    Validators.required,  
    Validators.minLength(3)  
  ]),  
  email: new FormControl('', [  
    Validators.required,  
    Validators.email  
  ])  
});
```

```
<form [formGroup]="form">  
  
  <input formControlName="username">
```



```
<p
*ngIf="form.get('username')?.errors?.['required']">
  Username is required
</p>

<input formControlName="email">
<p *ngIf="form.get('email')?.errors?.['email']">
  Invalid email
</p>

</form>
```

How Errors Work Internally

```
this.username.errors
```

👉 Output:

```
{
  "required": true
}
```

When Validation Runs

Angular validates automatically when:

- User types (value change)
- Field loses focus
- Form is submitted

Form-Level Validation

👉 You can validate **entire form**

Example:

- Password and Confirm Password must match

```
this.form = new FormGroup({  
  password: new FormControl(''),  
  confirmPassword: new FormControl('')  
}, { validators: passwordMatchValidator });
```

Why Angular Validation is Powerful

- Automatic validation
- Real-time feedback
- Easy error handling
- Scalable for large apps
- Works with both approaches

Template vs Reactive Validation

Feature	Template-driven	Reactive
Where rules defined	HTML	TypeScript
Control	Less	Full control
Scalability	Low	High

Interview One-Liner

Angular form validation works by attaching validators to form controls, automatically checking input values, and updating the control's validity and error state, which can be used to display feedback in the UI.

Final Intuition (Very Important)

👉 Validation = **rules + checking + status + UI feedback**

What are Custom Validators?

👉 Custom validators are user-defined functions used to validate form fields based on custom logic.

In simple words:

When Angular doesn't provide a validator for your requirement, you create your own.

Why Do We Need Custom Validators?

Built-in validators can only do basic checks:

- required
- email
- minLength

But real apps need more:

👉 Examples:

- Username should not contain spaces
- Password must contain special characters
- Age must be greater than 18
- Password and confirm password must match

Types of Custom Validators

There are **2 types**:

1 Control-level validator

👉 Works on a single field

2 Form-level validator

👉 Works on multiple fields (like password match)

1. Control-Level Custom Validator

Example: No spaces allowed

```
function noSpaceValidator(control: FormControl) {  
  if (control.value && control.value.includes(' ')) {  
    return { noSpace: true }; // invalid  
  }  
  return null; // valid  
}
```

✓ Usage

```
username = new FormControl('', [  
  Validators.required,  
  noSpaceValidator  
]);
```

Error Access

```
this.username.errors
```

👉 Output:

```
{  
  "noSpace": true  
}
```

HTML

```
<input [formControl]="username">  
  
<p *ngIf="username.errors?.['noSpace']">  
  Spaces are not allowed  
</p>
```

2. Form-Level Custom Validator

👉 Used when validation depends on multiple fields

Example: Password Match

```
function passwordMatchValidator(group: FormGroup) {  
  const password = group.get('password')?.value;  
  const confirm =  
group.get('confirmPassword')?.value;  
  
  if (password !== confirm) {  
    return { passwordMismatch: true };  
  }  
  return null;  
}
```

✓ Usage

```
this.form = new FormGroup({  
  password: new FormControl(''),  
  confirmPassword: new FormControl('')  
}, { validators: passwordMatchValidator });
```

HTML

```
<p *ngIf="form.errors?.['passwordMismatch']">  
  Passwords do not match  
</p>
```

Important Rule

👉 Custom validator must return:

- { errorName: true } → if invalid
- null → if valid

Async Custom Validators (Advanced)

👉 Used when validation needs API call

Example:

- Check if username already exists

```
function usernameExistsValidator(control:  
FormControl) {  
  return http.get('/api/check-username').pipe(  
    map(response => response.exists ?
```

```
{ usernameTaken: true } : null)
);
}
```

Where to Use Custom Validators

- Reactive Forms  (most common)
- Template-driven forms (possible but less common)

Built-in vs Custom Validators

Feature	Built-in	Custom
Provided by Angular		
Flexibility	Limited	Unlimited
Use case	Common rules	Business logic

Interview One-Liner

Custom validators in Angular are user-defined functions that implement custom validation logic and return an error object if validation fails, or null if it passes.

Final Intuition (Very Important)

- 👉 Built-in validators = ready-made rules
- 👉 Custom validators = your own business rules

What is HttpClient in Angular?

👉 **HttpClient** is a service in Angular used to communicate with a backend server via HTTP requests.

In simple words:

HttpClient = tool to send/receive data from APIs (backend)

Why Do We Need HttpClient?

Frontend (Angular) alone is not enough.

We need backend for:

- Fetching data (users, products)
- Saving data (forms)
- Updating data
- Deleting data

👉 HttpClient helps Angular talk to backend APIs.

Real-Life Example

Imagine:

- You click "**Login**"
- Angular sends data to server
- Server responds → success or error

👉 This communication is done using **HttpClient**

Where Does It Come From?

HttpClient is provided by:

```
import { HttpClient } from '@angular/common/http';
```

And you must import module:

```
import { HttpClientModule } from  
'@angular/common/http';
```

Basic Usage

1 Inject HttpClient

```
constructor(private http: HttpClient) {}
```

2 Make API Call

GET request

```
this.http.get('https://api.example.com/users')  
  .subscribe(data => {  
    console.log(data);  
  });
```

POST request

```
this.http.post('https://api.example.com/users', {  
  name: 'Lakshmi'  
}).subscribe(response => {  
  console.log(response);  
});
```

Common HTTP Methods

Method	Purpose
GET	Fetch data
POST	Create data
PUT	Update data
DELETE	Remove data

What HttpClient Returns

👉 HttpClient returns **Observable**

```
this.http.get(...)
```

👉 You must subscribe:

```
.subscribe()
```

Why Observable?

Because:

- API calls are async
- Data comes later

Angular uses **RxJS Observables** to handle this.

Example with Service (Best Practice)

👉 Always use a service

```
@Injectable()  
export class UserService {
```

```
constructor(private http: HttpClient) {}

getUsers() {
  return this.http.get('/api/users');
}
}

this.userService.getUsers().subscribe(data => {
  console.log(data);
});
```

Features of HttpClient

- Handles JSON automatically
- Supports interceptors
- Error handling
- Typed responses
- Easy testing

Important Points

- Always import HttpClientModule
- Always subscribe (or use async pipe)
- Use services instead of calling directly in components

Interview One-Liner

HttpClient in Angular is a service used to make HTTP requests to backend APIs and handle asynchronous data using **Observables**.

Final Intuition

👉 HttpClient = **bridge between frontend and backend**

How do you make API calls in Angular?

In Angular, API calls are made using **HttpClient**

It helps your Angular app communicate with backend APIs using HTTP methods like GET, POST, PUT, DELETE.

Step-by-Step Process (Real Flow)

Component → Service → HttpClient → Backend API → Response → UI

Step 1: Import HttpClientModule

👉 In your App Module:

```
import { HttpClientModule } from
 '@angular/common/http';

@NgModule({
  imports: [HttpClientModule]
})
export class AppModule {}
```

Step 2: Create a Service (Best Practice)

👉 Always use a service instead of calling API directly in component

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class UserService {

  constructor(private http: HttpClient) {}

  getUsers() {
    return this.http.get('/api/users');
  }

  addUser(user: any) {
    return this.http.post('/api/users', user);
  }
}
```

Step 3: Use Service in Component

```
constructor(private userService: UserService) {}

ngOnInit() {
  this.userService.getUsers().subscribe(data => {
    console.log(data);
  });
}
```

Common API Calls

GET (Fetch data)

```
this.http.get('/api/users');
```

POST (Send data)

```
this.http.post('/api/users', { name: 'Lakshmi' });
```

PUT (Update data)

```
this.http.put('/api/users/1', updatedData);
```

DELETE (Remove data)

```
this.http.delete('/api/users/1');
```

Why .subscribe()?

👉 HttpClient returns **Observable**

```
this.http.get(...).subscribe(response => {  
  console.log(response);  
});
```

👉 Because API calls are **asynchronous** (data comes later)

Error Handling (Important)

```
this.http.get('/api/users').subscribe({  
  next: data => console.log(data),  
  error: err => console.error(err)  
});
```

Best Practices (Interview + Real Projects)

✓ Use **services** for API calls

✓ Use **typed models**

```
getUsers(): Observable<User[]> {  
  return this.http.get<User[]>('/api/users');  
}
```

✓ Handle errors properly

✓ Use **interceptors** for auth tokens (advanced)

✓ Avoid business logic in components

Interview One-Liner

In Angular, API calls are made using **HttpClient** inside a service, which sends **HTTP** requests and returns **Observables** that are subscribed to in components to handle asynchronous responses.

What is the difference between **HttpClient** and **fetch()**?

👉 Both **HttpClient** and **Fetch API** are used to make HTTP requests.

But:

HttpClient is Angular-specific and powerful
fetch() is a browser-native basic API

Key Difference (Simple View)

Feature	HttpClient (Angular)	fetch() (JS)
Type	Angular service	Browser API
Return type	Observable	Promise
JSON handling	Automatic	Manual (.json())
Error handling	Built-in	Manual
Interceptors	✅ Yes	❌ No
RxJS support	✅ Yes	❌ No
Angular integration	Full	None

1. Return Type Difference (Very Important)

HttpClient → Observable

```
this.http.get('/api/users').subscribe(data => {  
  console.log(data);  
});
```

👉 Can:

- Cancel request
- Retry
- Chain operators

fetch() → Promise

```
fetch('/api/users')  
  .then(res => res.json())  
  .then(data => console.log(data));
```

👉 Simpler, but:

- No cancellation (easily)
- Limited features

2. JSON Handling

HttpClient

```
this.http.get('/api/users')
```

👉 Automatically converts JSON → object

fetch()

```
fetch('/api/users')  
  .then(res => res.json())
```

👉 You must manually parse

3. Error Handling

HttpClient

```
this.http.get('/api').subscribe({  
  error: err => console.log(err)  
});
```

👉 Handles HTTP errors properly

fetch()

```
fetch('/api')
  .then(res => {
    if (!res.ok) throw new Error('Error');
  });
```

👉 You must check manually

4. Interceptors (Big Advantage)

HttpClient

👉 Supports interceptors (VERY IMPORTANT in real apps)

- Add auth token
- Log requests
- Handle global errors

fetch()

❌ No built-in interceptor system

5. Angular Ecosystem Integration

HttpClient

- Works with Angular DI
- Works with services
- Works with RxJS
- Easy testing

fetch()

- Plain JavaScript
- No Angular features

Real-World Understanding

👉 In Angular apps:

- Use **HttpClient**  (standard)
- Avoid `fetch()` unless special case

When Would You Use fetch()?

- Non-Angular apps
- Simple scripts
- Very lightweight use

Interview One-Liner

HttpClient is an Angular service that uses **Observables** and provides advanced features like interceptors and automatic JSON handling, while **fetch()** is a native JavaScript API that uses **Promises** and requires manual handling of responses and errors.

Final Intuition

👉 HttpClient = **enterprise-level tool**

👉 fetch() = **basic tool**

How do you handle HTTP errors in Angular?

👉 In Angular, HTTP errors are handled using **HttpClient** with:

- subscribe() error callback
- RxJS operators like catchError
- Global handling using Interceptors

1. Basic Error Handling (Using subscribe)

👉 The simplest way

```
this.http.get('/api/users').subscribe({
  next: data => console.log(data),
  error: err => {
    console.error('Error occurred:', err);
  }
});
```

What You Get in error

```
err.status      // HTTP status (404, 500)
err.message     // error message
err.error       // backend response
```

2. Using catchError (Best Practice)

👉 Use RxJS operator for cleaner handling

```
import { catchError } from 'rxjs/operators';
import { throwError } from 'rxjs';

getUsers() {
  return this.http.get('/api/users').pipe(
    catchError(error => {
      console.log('Handled in service:', error);
      return throwError(() => error);
    })
  );
}
```

In Component

```
this.userService.getUsers().subscribe({
  next: data => console.log(data),
  error: err => console.log('Handled in component:', err)
});
```

3. Centralized Error Handling (Interceptor) 🚀

👉 Best for real-world apps

Create Interceptor

```
@Injectable()
export class ErrorInterceptor implements
HttpInterceptor {
```

```

    intercept(req: HttpRequest<any>, next: HttpHandler)
    {
        return next.handle(req).pipe(
            catchError((error: HttpResponse) => {

                if (error.status === 401) {
                    console.log('Unauthorized');
                } else if (error.status === 500) {
                    console.log('Server error');
                }

                return throwError(() => error);
            })
        );
    }
}

```

Register Interceptor

```

providers: [
    { provide: HTTP_INTERCEPTORS, useClass:
ErrorInterceptor, multi: true }
]

```

Why Interceptors Are Powerful

👉 Handle errors **in one place**

- Authentication errors
- Logging
- Global messages

- Redirect to login

4. User-Friendly Error Messages

👉 Don't show raw errors to users

```
catchError(error => {  
  let message = 'Something went wrong';  
  
  if (error.status === 404) {  
    message = 'Data not found';  
  }  
  
  return throwError(() => message);  
})
```

Error Types You Should Handle

Status Code	Meaning
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
500	Server error

Real-World Flow

API Call → Error occurs → catchError / interceptor → Handle → Show message

Best Practices

- ✓ Use `catchError` in services
- ✓ Use **interceptors for global errors**
- ✓ Show user-friendly messages
- ✓ Log errors for debugging
- ✓ Don't put heavy logic in components

Interview One-Liner

HTTP errors in Angular are handled using `HttpClient` through subscribe error callbacks, RxJS `catchError` operators, and globally using HTTP interceptors for centralized error management.

Final Intuition

- 👉 Local handling → specific cases
- 👉 Interceptor → global control

What is an HTTP Interceptor?

👉 An **HTTP Interceptor** in Angular is a mechanism that allows you to:

Intercept and modify every HTTP request and response globally

It works with **`HttpClient`**

Simple Understanding

- Before request goes to server → you can change it
- After response comes → you can handle it

👉 It acts like a **middle layer between Angular app and backend**

Flow Visualization

Component → Service → HttpClient → Interceptor → Server

↓

Component ← Service ← HttpClient ← Interceptor ← Response

Basic Example

```
@Injectable()
export class MyInterceptor implements HttpInterceptor
{
    intercept(req: HttpRequest<any>, next: HttpHandler)
    {
        // Modify request
        const modifiedReq = req.clone({
            setHeaders: {
                Authorization: 'Bearer token'
            }
        });

        // Pass request forward
        return next.handle(modifiedReq);
    }
}
```

```
}  
}
```

Register Interceptor

```
providers: [  
  { provide: HTTP_INTERCEPTORS, useClass:  
    MyInterceptor, multi: true }  
]
```

Key Points

- ✓ Runs for every HTTP request
- ✓ Can modify request and response
- ✓ Works globally
- ✓ Uses `req.clone()` (requests are immutable)

Interview One-Liner

An HTTP interceptor in Angular is a feature that intercepts HTTP requests and responses globally, allowing developers to modify them or handle logic such as authentication and error handling.

Why are HTTP Interceptors Used in Angular?

👉 Interceptors are used to:

Handle common HTTP logic in one central place instead of repeating it in every API call

They work with **HttpClient**

Core Problem (Without Interceptors)

Imagine you have 20 API calls:

```
this.http.get('/api/users', {  
  headers: { Authorization: 'Bearer token' }  
});
```

👉 You would repeat:

- Auth token
- Error handling
- Logging

❌ Leads to duplicate code

❌ Hard to maintain

Solution → Interceptors

👉 Write logic **once** → applies to all API calls automatically

Main Uses of Interceptors

1 Add Authentication Token (Most Important 🙌)

```
req.clone({  
  setHeaders: {  
    Authorization: 'Bearer token'  
  }  
});
```

- ✓ Automatically attaches token to every request
- ✓ Used in JWT authentication

2 Global Error Handling

```
catchError(error => {  
  if (error.status === 401) {  
    // redirect to login  
  }  
});
```

- ✓ Handle all API errors in one place
- ✓ Cleaner code

3 Logging Requests & Responses

```
console.log(req.url);
```

- ✓ Debugging
- ✓ Monitoring API calls

4 Show Loader / Spinner

- 👉 Start loader when request starts
- 👉 Stop when response comes

- ✓ Improves user experience

5 Modify Requests

- Add headers

- Add query params
- Change URL

6 Retry Failed Requests

👉 Automatically retry API calls if needed

Real-World Flow

```
Component → Service → HttpClient → Interceptor → API
                                   ↓
Component ← Service ← HttpClient ← Interceptor ←
Response
```

Without vs With Interceptor

Without Interceptor	With Interceptor
Duplicate code	Centralized logic
Hard to manage	Easy to maintain
Error handling everywhere	Global error handling
Token added manually	Automatic

Key Benefit

👉 Write once → apply everywhere

Interview One-Liner

Interceptors are used in Angular to centralize common HTTP logic such as adding authentication headers, handling errors,

logging, and modifying requests or responses, improving code maintainability and reusability.

Final Intuition

Interceptor = **middleware for all HTTP calls**